

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

DANIEL TABORDA AFONSO

**REDES CONVOLUCIONAIS EM GRAFOS APLICADO À ESTIMATIVA DE
ESFORÇO DE SOFTWARE POR ANALOGIA A PARTIR DE REQUISITOS
TEXTUAIS**

PATO BRANCO

2025

DANIEL TABORDA AFONSO

**REDES CONVOLUCIONAIS EM GRAFOS APLICADO À ESTIMATIVA DE
ESFORÇO DE SOFTWARE POR ANALOGIA A PARTIR DE REQUISITOS
TEXTUAIS**

**GRAPH CONVOLUTIONAL NETWORKS APPLIED TO SOFTWARE EFFORT
ESTIMATION BY ANALOGY FROM TEXTUAL REQUIREMENTS**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção
do título de Bacharel em Engenharia de
Computação do Curso de Bacharelado em
Engenharia de Computação da Universidade
Tecnológica Federal do Paraná.

Orientador: Prof^ª. Dr^ª. Eliane Maria De Bortoli
Fávero

PATO BRANCO

2025



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

DANIEL TABORDA AFONSO

**REDES CONVOLUCIONAIS EM GRAFOS APLICADO À ESTIMATIVA DE
ESFORÇO DE SOFTWARE POR ANALOGIA A PARTIR DE REQUISITOS
TEXTUAIS**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção
do título de Bacharel em Engenharia de
Computação do Curso de Bacharelado em
Engenharia de Computação da Universidade
Tecnológica Federal do Paraná.

Data de aprovação: 26/novembro/2025

Eliane Maria de Bortoli Fávero
Doutor
Universidade Tecnológica Federal do Paraná

Dalcimar Casanova
Doutor
Universidade Tecnológica Federal do Paraná

Kathya Silvia Collazos Linares
Doutor
Universidade Tecnológica Federal do Paraná

PATO BRANCO
2025

Dedico este trabalho aos meus pais, Valdocir Afonso e Leonilda Taborda Afonso, em gratidão pelo apoio incondicional, pelos conselhos e por nunca deixarem que eu desistisse dos meus sonhos.

RESUMO

Este trabalho explora a aplicação de métodos avançados de aprendizado de máquina e processamento de linguagem natural para a estimativa de esforço por analogia em projetos de software. A estimativa de esforço de software é uma tarefa crucial que impacta diretamente no planejamento, no prazo e no orçamento dos projetos. Tradicionalmente, métodos baseados em analogia ou julgamento de especialistas têm sido utilizados, porém, esses métodos muitas vezes carecem de precisão devido à subjetividade e à complexidade envolvidas. Este estudo justifica-se pela necessidade de melhorar a precisão dessas estimativas por meio do uso de modelos de *deep learning*. O objetivo principal é implementar, treinar e avaliar um modelo BertGCN, que combina as capacidades do modelo de linguagem BERT com Redes Convolucionais em Grafos (GCNs). A metodologia inclui a preparação de um conjunto de dados de requisitos de software, a obtenção de *embeddings* com o BERT, a construção do modelo BertGCN e a comparação de seu desempenho com o modelo SE³M utilizando métricas como Erro Absoluto Médio, do Inglês *Mean Absolute Error* (MAE), Erro Quadrático Médio, do Inglês *Mean Squared Error* (MSE) e Erro Absoluto Mediano, do Inglês *Median Absolute Error* (MdAE). A conclusão destaca a importância da pesquisa e do uso de modelos de *deep learning*, como o BertGCN, na estimativa de esforço de software.

Palavras-chave: processamento de linguagem natural; estimativa de esforço de software; redes convolucionais em grafo; bertgcn; aprendizado de máquina.

ABSTRACT

This work explores the application of advanced machine learning and natural language processing methods for software effort estimation by analogy in software projects. Effort estimation is a crucial task that directly impacts project planning, deadlines, and budgets. Traditionally, methods based on analogy or expert judgment have been used, however, these methods often lack accuracy due to the subjectivity and complexity involved. This study is justified by the need to improve the accuracy of these estimates through the use of deep learning models. The main objective is to implement, train, and evaluate a BertGCN model, which combines the capabilities of the BERT language model with Graph Convolutional Networks (GCNs). The methodology includes preparing a dataset of software requirements, obtaining embeddings with BERT, constructing the BertGCN model, and comparing its performance with the SE³M model using metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and Median Absolute Error (Mdae). The conclusion highlights the importance of the research and the use of deep learning models, such as BertGCN, in software effort estimation.

Keywords: natural language processing; effort estimation; graph convolutional networks; bertgcn; machine learning.

LISTA DE FIGURAS

Figura 1 – Exemplo de cidades e suas conexões representadas por um grafo.	20
Figura 2 – Exemplo de um grafo e sua matriz de adjacência	20
Figura 3 – Exemplo de grafo com características iniciais dos nós.	24
Figura 4 – Exemplo de grafo com representação de suas características atualiza- das após operação de convolução.	26
Figura 5 – Exemplos de plot 3D de word vectors.	27
Figura 6 – Exemplo de uso do mecanismo de auto-atenção.	28
Figura 7 – Fluxograma metodológico da pesquisa, contemplando a geração dos <i>corpora</i> , a construção dos grafos heterogêneos e o treinamento e ava- liação do modelo BertGCN.	48
Figura 8 – Exemplo de grafo heterogêneo documento–palavra utilizado na cons- trução do TextGCN/BertGCN, em que nós azuis representam requi- sitos (documentos), nós vermelhos representam palavras, as arestas documento–palavra são ponderadas por TF–IDF e as arestas palavra– palavra por PPMI.	52
Figura 9 – Fluxo de pré-processamento, tokenização e geração de <i>embeddings</i> BERT para cada requisito.	52
Figura 10 – Exemplo de uma matriz de características inicial.	53
Figura 11 – Histograma representando a medida de Story Point em relação à sua frequência no Corpus	62
Figura 12 – Visualização do grafo heterogêneo construído para o Corpus_1. Grafo com 23.313 documentos, 11.326 palavras e 687.420 arestas	68

LISTA DE TABELAS

Tabela 1 – Descrição e número de requisitos textuais por projeto.	47
Tabela 2 – Estatísticas descritivas de <i>Story Points</i> do conjunto de dados.	61
Tabela 3 – Análise de outliers pelo método IQR	63
Tabela 4 – Teste de normalidade (<i>D’Agostino-Pearson</i>)	63
Tabela 5 – Estatísticas descritivas de <i>story points</i> por projeto	64
Tabela 6 – Teste ANOVA para heterogeneidade entre projetos	64
Tabela 7 – Estatísticas da análise textual dos requisitos	65
Tabela 8 – Exemplo de transformações aplicadas no pré-processamento	67
Tabela 9 – Resultados do Experimento E1 (Multi-repositórios) - Corpus_1 e Corpus_2	69
Tabela 10 – Resultados do Experimento E2 (Cross-repository LOPO) - Corpus_1 e Corpus_2	70
Tabela 11 – Valores do MAE obtidos em cada subconjunto no experimento E2 dos modelos BertGCN (Corpus_1 e Corpus_2) e dos experimentos análogos realizados com SE ³ M e Z-SE ² , com dados dos respectivos projetos utilizados para avaliação. Também são apresentados o identificador do projeto (ID), o número de requisitos por projeto (Requisitos/projeto), a média (M_E) e o desvio padrão (DP) do esforço por requisito em cada projeto. Quanto menor o valor do MAE, melhor o resultado. Os menores valores de MAE de cada projeto foram destacados em negrito. . . .	70
Tabela 12 – Erro Absoluto Médio (MAE), Erro Quadrático Médio (MSE) e Mediana dos Erros Absolutos (MdAE) obtidos ao aplicar o BertGCN, comparados aos modelos SE ³ M, Z-SE ² e Deep-SE. Para todas as métricas, quanto menor o valor melhor o resultados.	72

LISTA DE QUADROS

Quadro 1 – Trabalhos relacionados à estimativa de esforço	45
Quadro 2 – Materiais Utilizados no Desenvolvimento do trabalho	46

LISTA DE ABREVIATURAS E SIGLAS

Siglas

AM	Aprendizado de Máquina
ANN	Redes Neurais Artificiais, do Inglês <i>Artificial Neural Networks</i>
CNN	Redes Neurais Convolucionais, do inglês <i>Convolutional Neural Networks</i>
DL	<i>Deep Learning</i>
EES	Estimativa de Esforço de Software
GCN	Redes Convolucionais em Grafos, do inglês <i>Graph Convolutional Networks</i>
GPT	<i>Transformer</i> Pré-treinado Generativo, do inglês <i>Generative Pre-trained Transformer</i>
IA	Inteligência Artificial
IQR	Amplitude interquartil, do Inglês <i>Interquartile Range</i>
LLM	<i>Large Language Model</i>
MAE	Erro Absoluto Médio, do Inglês <i>Mean Absolute Error</i>
MdAE	Erro Absoluto Mediano, do Inglês <i>Median Absolute Error</i>
MSE	Erro Quadrático Médio, do Inglês <i>Mean Squared Error</i>
PLN	Processamento de Linguagem Natural
TF	Frequência do Termo, do Inglês <i>Term Frequency</i>
TF-IDF	<i>Term Frequency-Inverse Document Frequency</i>

Acrônimos

BERT	<i>Bidirectional Encoder Representations from Transformers</i>
EESA	Estimativa de Esforço de Software por Analogia
GloVe	<i>Global Vectors</i>
LOPO	<i>Leave One Project Out</i>
SE ³ M	Modelo de Embedding para Estimativa do Esforço de Software, do Inglês <i>Software Effort Estimation Embedding Model</i>

Word2Vec	<i>Word to vector</i>
Z-SE ²	<i>Zero-shot Software Effort Estimator</i>

SUMÁRIO

1	INTRODUÇÃO	13
1.1	Considerações iniciais	13
1.2	Objetivos	15
1.2.1	Objetivo geral	15
1.2.2	Objetivos específicos	15
1.3	Justificativa	15
1.4	Estrutura do trabalho	16
2	REFERENCIAL TEÓRICO	17
2.1	Processamento de Linguagem Natural	17
2.2	Estimativa de Esforço de Software	17
2.2.1	Estimativa de Esforço de Software por Analogia (EESA)	18
2.3	Grafos	19
2.4	Redes Neurais Artificiais (ANNs)	21
2.5	Redes Convolucionais em Grafos (GCNs)	22
2.6	Operação de Convolução em Grafos	23
2.6.1	Definição e Formalização	23
2.6.2	Exemplo Prático	23
2.7	Representação Textual	26
2.7.1	<i>Word Embeddings</i>	26
2.7.2	Modelos de Representação Pré-treinados	27
2.7.2.1	<u>BERT</u>	29
2.7.3	Representação Textual por Grafos	29
2.8	Classificação Textual por GCNs	30
2.9	Aprendizado Transdutivo	31
2.9.1	Fundamentos Teóricos	31
2.9.2	Diferenças entre Aprendizado Indutivo e Transdutivo	32
2.9.3	Aprendizado Semi-Supervisionado e Transdutivo	32
2.9.4	Propagação de Rótulos em Grafos	33
2.9.5	Aplicações em Processamento de Linguagem Natural	33
2.9.6	Vantagens e Limitações	33

2.10	BertGCN	34
2.11	Métricas e Medidas Estatísticas para Análise Exploratória de Dados	35
2.11.1	Medidas de Tendência Central	35
2.11.2	Medidas de Dispersão	36
2.11.3	Medidas de Forma da Distribuição	36
2.11.4	Detecção de <i>Outliers</i>	37
2.11.5	Testes de Normalidade	37
2.11.6	Teste de Heterogeneidade entre Grupos: ANOVA	38
2.11.7	Análise de Correlação	38
2.11.8	Implicações para Modelagem de Aprendizado de Máquina	39
2.11.9	Métricas de Avaliação para Modelos de Regressão	40
2.11.9.1	Erro Absoluto Médio	40
2.11.9.2	Erro Quadrático Médio	40
2.11.9.3	Erro Absoluto Mediano	41
3	TRABALHOS RELACIONADOS	42
3.1	Modelos de Estimativa de Esforço de Software por Analogia (EESA)	42
3.2	Classificação de Texto com GCNs	43
3.3	Posicionamento do Trabalho Proposto	44
4	MATERIAIS E MÉTODOS	46
4.1	Materiais	46
4.1.1	Base de Dados	47
4.2	Método	47
4.2.1	Análise Exploratória dos Dados	47
4.2.2	Pré-processamento dos Dados	49
4.2.2.1	Corpus_1: Pré-processamento Tradicional	49
4.2.2.2	Corpus_2: Refinamento via Modelo de Linguagem	50
4.2.3	Representação via BertGCN	51
4.2.4	Particionamento dos dados	54
4.2.4.1	Experimentos Conduzidos	54
4.2.4.2	Considerações sobre Aprendizado Transdutivo	55
4.2.5	Treinamento do Modelo	56
4.2.5.1	Arquitetura do Modelo	56

4.2.5.1.1	<i>Escolha de $\lambda = 1.0$: Foco exclusivo na GCN</i>	56
4.2.5.2	<u>Configuração de Hiperparâmetros</u>	57
4.2.5.3	<u>Processo de Treinamento</u>	58
4.2.5.3.1	<i>Treinamento das Camadas GCN</i>	58
4.3	Inferência da Estimativa de Esforço de Software	59
4.4	Métricas de Avaliação	59
4.4.1	Resultados e Análise	59
5	RESULTADOS E DISCUSSÃO	60
5.1	Análise Exploratória dos Dados	60
5.1.1	Estatísticas Descritivas e Distribuição de <i>Story Points</i>	61
5.1.2	Detecção de <i>Outliers</i>	62
5.1.3	Teste de Normalidade	63
5.1.4	Heterogeneidade entre Projetos	63
5.1.5	Análise Textual dos Requisitos	65
5.1.5.1	<u>Correlação entre Comprimento Textual e <i>Story Points</i></u>	66
5.2	Pré-processamento dos Dados	66
5.3	Estrutura do Grafo Heterogêneo	67
5.3.1	Visualização da Estrutura do Grafo	67
5.4	Resultados dos Experimentos	68
5.4.1	Experimento E1: Multi-repositórios	69
5.4.2	Experimento E2: <i>Cross-repository</i>	69
5.4.3	Análise Comparativa com a Literatura	72
5.4.4	Discussão dos Resultados	73
5.4.4.1	<u>Desempenho do BertGCN em Relação aos Modelos SE³M e Z-SE²</u>	73
5.4.4.2	<u>Impacto do Refinamento via <i>GPT-5 mini</i></u>	73
5.4.4.3	<u>Heterogeneidade e Generalização <i>Cross-Repository</i></u>	74
5.4.4.4	<u>Limitações Metodológicas e Trabalhos Futuros</u>	74
6	CONCLUSÃO	76
	REFERÊNCIAS	77
	GLOSSÁRIO	84

1 INTRODUÇÃO

1.1 Considerações iniciais

A Inteligência Artificial (IA) aplicada a diversos contextos tem sido foco de muitos pesquisadores nas últimas décadas. Com o avanço da computação em termos de hardware, o poder de processamento das máquinas aumentou significativamente, tornando-se mais acessível. Uma subárea da IA que se destacou é a de Processamento de Linguagem Natural (PLN), que oferece um vasto campo de pesquisa devido à sua diversidade de aplicações e surgimento de métodos inteligentes para representação e classificação textual, como por exemplo o surgimento de modelos pré-treinados (ex. BERT, GPT) (Devlin *et al.*, 2018), (Radford *et al.*, 2019).

PLN é a área responsável pelo processamento da linguagem humana. De acordo com Gu *et al.* (2018), PLN permite que computadores realizem uma ampla gama de tarefas relacionadas à linguagem natural, como segmentação de palavras, extração de informações, reconhecimento de entidades nomeadas, marcação de partes do discurso, desambiguação de sentido de palavras, reconhecimento de fala, tradução automática, sumarização, resposta a perguntas, compreensão de linguagem natural, reconhecimento óptico de caracteres e recuperação de informações. Segundo Nadkarni, Ohno-Machado e Chapman (2011), o objetivo final do PLN é extrair significado (ou semântica) do texto, e métodos de aprendizado de máquina tornaram-se proeminentes, oferecendo bons resultados para essa finalidade.

Com novas ferramentas e modelos de aprendizado de máquina, PLN tornou-se uma área promissora para a pesquisa e desenvolvimento, especialmente com o avanço nas aplicações de *deep learning*¹. Surgiram, então, as Redes Neurais Convolucionais, do inglês *Convolutional Neural Networks* (CNN), inicialmente aplicadas ao processamento de imagens, mas com grande potencial também para o PLN. Segundo (Gu *et al.*, 2018), uma CNN é um método de rede neural de várias camadas para aprender as características hierárquicas dos dados.

A classificação de textos é um problema fundamental em PLN, com diversas aplicações em mídias sociais, assistência médica e bioinformática (Malekzadeh *et al.*, 2021). Essa tarefa é realizada com métodos de aprendizado de máquina aplicados ao treinamento de modelos inteligentes para classificar textos de acordo com rótulos ou características específicas. Para isso, os textos são convertidos para uma representação compreensível pelo computador, processo que pode ocorrer a nível de palavras ou frases.

Diversos métodos de representação de textos têm sido utilizados ao longo dos estudos em PLN. Entre os mais simples está o *bag-of-words*. Um método mais avançado para representar textos, evitando a esparsidade do vetor de características, são os *embeddings*. Exis-

¹ *Deep learning* é uma subárea do aprendizado de máquina que utiliza redes neurais com múltiplas camadas para aprender representações de dados em diferentes níveis de abstração. Essas redes são particularmente eficazes em tarefas como reconhecimento de imagem e fala, além de processamento de linguagem natural (Goodfellow; Bengio; Courville, 2016; LeCun; Bengio; Hinton, 2015)

tem dois tipos principais de estratégias para gerar *embeddings*: modelos sequenciais, como Word2Vec e GloVe (Oliveira *et al.*, 2022), que utilizam a sequência de tokens para agregar informações contextuais; e modelos bidirecionais (Devlin *et al.*, 2018), que analisam o texto de forma abrangente, utilizando mecanismos avançados de atenção, como os contidos nos modelos pré-treinados BERT, GPT-2 e GPT-3.

Novas formas de representação textual surgiram recentemente, e têm demonstrado bons resultados. Nesse contexto, uma área emergente de pesquisa é a das redes neurais de grafos, que se mostraram eficazes em tarefas com estruturas relacionais complexas. Embora todas as redes neurais possam ser vistas como grafos, pois consistem em nós (neurônios) e arestas (conexões sinápticas), as Redes Convolucionais em Grafos, do inglês *Graph Convolutional Networks* (GCN) diferenciam-se por operar diretamente sobre grafos onde os nós representam entidades e as arestas representam relações entre essas entidades. Essas redes preservam as informações da estrutura global de um grafo em seus *embeddings*, oferecendo uma representação rica e detalhada dos dados textuais (Battaglia *et al.*, 2018; Cai; Zheng; Chang, 2018). Yao, Mao e Luo (2018) propuseram um método para classificação de textos baseado na geração de um único grande grafo para todo o conjunto de textos, em que palavras e documentos são nós do grafo. Esse grafo é modelado com uma GCN (Kipf; Welling, 2017), onde as arestas entre nós de palavras são construídas por informações de coocorrência de palavras, e as arestas entre nós de palavras e documentos são construídas usando a Frequência do Termo, do Inglês *Term Frequency* (TF) e a frequência de documento inversa (TF-IDF). Assim, o problema de classificação de textos é transformado em um problema de classificação de nós. Segundo os autores, esse método pode apresentar um ótimo desempenho na classificação, mesmo com um pequeno volume de textos rotulados.

Considerando esse contexto, este trabalho propõe a aplicação de um modelo de representação de texto que combina o modelo pré-treinado BERT e redes de grafos convolucionais, o que é chamado de BertGCN (Lin *et al.*, 2022) no problema de estimativa de esforço de software, a exemplo de Fávero, Casanova e Pimentel (2022). Nesse contexto, esforço de software pode ser entendido como uma unidade de medida de tempo para o ciclo de desenvolvimento de um determinado requisito. O modelo BertGCN utiliza BERT para gerar *embeddings* a partir das sentenças de requisitos textuais, que serão posteriormente integrados em uma rede convolucional de grafos (GCN). Essa combinação permite capturar tanto a semântica rica dos textos quanto as relações estruturais dos dados. Com isso, há a hipótese de que os resultados podem ser melhorados em relação a abordagem de Fávero, Casanova e Pimentel (2022) para Estimativa de Esforço de Software por Analogia (EESA).

Para verificar essa possível melhoria nos resultados, a base de dados utilizada neste trabalho é a mesma de Fávero, Casanova e Pimentel (2022), que propôs o Modelo de Embedding para Estimativa do Esforço de Software, do Inglês *Software Effort Estimation Embedding Model* (SE³M). Essa base consiste de histórias de usuário de diversos projetos de software de

código aberto, as quais são acompanhadas de seus respectivos *story points* realizados. **Desta forma, esse trabalho consiste em responder à seguinte questão de pesquisa:**

- **Qual a eficácia do BertGCN aplicado para estimativa de esforço de software, quando comparado ao modelo SE³M (Fávero; Casanova; Pimentel, 2022)?**

Os resultados deste trabalho objetivam contribuir com a comunidade acadêmica e científica nas áreas de PLN e Engenharia de Software, incentivando melhorias nos métodos de estimativa existentes e novas aplicações na indústria de software.

1.2 Objetivos

1.2.1 Objetivo geral

- Avaliar a eficácia do modelo BertGCN, baseado em redes convolucionais em grafos, na estimativa de esforço de software por analogia a partir de requisitos textuais.

1.2.2 Objetivos específicos

- Realizar a análise exploratória da base de dados;
- Comparar empiricamente o desempenho preditivo dos modelos BertGCN e SE³M por meio de métricas de erro (MAE, MSE, MdAE), discutindo implicações para a estimativa de esforço de software por analogia.

1.3 Justificativa

A estimativa de esforço é uma etapa crucial no ciclo de desenvolvimento de software, pois permite aos gestores e desenvolvedores planejarem adequadamente os recursos necessários para a conclusão de um projeto dentro do prazo e do orçamento previstos.

No entanto, a estimativa de esforço muitas vezes é uma tarefa desafiadora, especialmente em projetos de grande escala com variados contextos de requisitos. Nesse sentido, a utilização de abordagens automatizadas para a estimativa de esforço pode trazer benefícios significativos para as empresas de desenvolvimento de software, tais como otimização de recursos, identificação de possíveis atrasos ou desvios no cronograma e melhoria da qualidade do planejamento e da tomada de decisão.

Assim, o desenvolvimento de um modelo de estimativa de esforço de software baseado em técnicas avançadas de processamento de linguagem natural e representação de dados estruturados não só contribuirá para a academia e a pesquisa científica, mas também para o avanço prático da indústria de desenvolvimento de software. Essa aplicação direta dos conheci-

mentos adquiridos durante o curso demonstra a relevância e o impacto potencial deste trabalho no contexto da Engenharia de Computação.

1.4 Estrutura do trabalho

Esse trabalho segue a seguinte estrutura: o segundo capítulo revisa os principais conceitos para a base teórica do trabalho. O terceiro capítulo apresenta os trabalhos relacionados. O quarto capítulo descreve os materiais utilizados e detalha o método adotado. No quinto capítulo, são apresentados e discutidos os resultados obtidos com as análises e experimentos. E no sexto as conclusões sobre o trabalho.

2 REFERENCIAL TEÓRICO

2.1 Processamento de Linguagem Natural

Processamento de Linguagem Natural (PLN) é um campo interdisciplinar que combina técnicas da linguística computacional, inteligência artificial e mecanismos cognitivos para permitir que computadores manipulem a linguagem humana de forma inteligente (Jurafsky; Martin, 2019). Esse tema multidisciplinar combina conhecimentos e técnicas da linguística, ciência da computação e inteligência artificial para criar sistemas capazes de lidar com textos e discursos de maneira inteligente.

Historicamente, cientistas têm se dedicado ao desenvolvimento de sistemas capazes de processar e compreender a linguagem humana. No entanto, foi com o avanço da inteligência artificial, especialmente com o surgimento de técnicas baseadas em machine learning (ML) e deep learning (DL), que o PLN deu grandes saltos em termos de desempenho e aplicabilidade.

O PLN abrange uma ampla gama de tarefas e aplicações, incluindo reconhecimento de fala, tradução automática, sumarização de texto, análise de sentimentos, geração de texto, entre outras. Essas tarefas têm aplicações práticas em diversas áreas, desde assistentes virtuais e sistemas de recomendação (Jannach *et al.*, 2021), até análise de mídias sociais (Farzindar; Inkpen; Hirst, 2015) e processamento de documentos jurídicos (Zhong *et al.*, 2020).

2.2 Estimativa de Esforço de Software

A Estimativa de Esforço de Software (EES) é uma das etapas cruciais no desenvolvimento de software. Para compreendê-la, é essencial revisar o conceito de esforço. O esforço refere-se à quantidade de unidades de trabalho necessárias para completar uma tarefa ou atividade específica (por exemplo, uma funcionalidade de software), podendo ser mensurado em homem/dia, homem/hora, entre outras unidades. Quando discutimos o custo de software, nos referimos ao valor monetário correspondente às unidades de medida empregadas em seu desenvolvimento (Abdukalykov, 2011).

Ao longo das últimas décadas, várias classificações para modelos de Estimativa de Esforço de Software (EES) foram propostas, cada uma com pequenas diferenças baseadas na perspectiva dos autores. Algumas dessas classificações são apresentadas a seguir.

Os modelos de EES podem ser categorizados em algorítmicos paramétricos e não-algorítmicos. Os modelos algorítmicos utilizam métodos matemáticos ou algorítmicos aplicados a atributos do projeto para calcular a estimativa. Exemplos desses modelos incluem o *Constructive Cost Model II* (COCOMO II) (Boehm; Abts; Chulani, 2000), e a Análise de Pontos de Função (Choi; Park; Sugumaran, 2012). Por outro lado, os modelos não-algorítmicos são baseados em técnicas de Aprendizado de Máquina (Aprendizado de Máquina (AM)), que dependem de da-

dos históricos para gerar modelos capazes de aprender a partir desses dados (Manikavelan; Ponnusamy, 2014; Shivhare; Rath, 2014).

Os modelos de EES também podem ser classificados conforme suas técnicas, dividindo-se em três categorias (Shepperd, 2007):

1. centradas em humanos (ex. julgamento de especialistas);
2. baseadas em modelos, incluindo modelos algorítmicos/paramétricos (ex. COCOMO);
3. técnicas de predição induzidas.

Embora as técnicas centradas em humanos sejam amplamente utilizadas, especialmente em métodos ágeis, elas apresentam problemas de subjetividade, muitas vezes resultando em estimativas distorcidas devido ao excesso de otimismo. A estimativa baseada em modelo é uma alternativa mais objetiva, oferecendo métodos reproduzíveis para gerar estimativas. As técnicas de predição induzidas utilizam modelos de AM (ex. regressão linear, redes neurais, analogia) para formar a base necessária para a estimativa. Nesses casos, os dados usados para treinar os modelos devem ser independentes dos dados utilizados em modelos algorítmicos/paramétricos (ex. COCOMO, Pontos por Função) (Kocaguneli *et al.*, 2011; Menzies *et al.*, 2006; Shepperd, 2007).

A Estimativa de Esforço de Software por Analogia (EESA) tem sido amplamente adotada, tratando do processo de identificar um ou mais projetos históricos semelhantes ao projeto em desenvolvimento e derivar a estimativa a partir deles. Esse método foi proposto pela primeira vez em 1977 por Sternberg (1977) como uma alternativa viável às opiniões de especialistas e às estimativas algorítmicas de esforço (Chiu; Huang, 2007).

2.2.1 Estimativa de Esforço de Software por Analogia (EESA)

A EESA, também conhecida como estimativa baseada em analogia, é um método não-paramétrico que utiliza dados de projetos anteriores para estimar o esforço necessário para um novo projeto. Esta técnica é classificada por alguns autores (Idri; Amazal; Abran, 2015) como uma técnica de AM, uma vez que sua aplicação tem sido automatizada por meio do uso de modelos de AM. Além disso, os modelos de AM, especialmente aqueles que utilizam *Deep Learning* (DL) devido à sua capacidade de lidar com não linearidades, têm atraído a atenção de pesquisadores no campo da EES. Esses modelos são capazes de mapear a complexa relação entre esforço e atributos de software (fatores de custo), especialmente quando essa relação não é linear e não segue um padrão predeterminado (Idri; Amazal; Abran, 2015; Kocaguneli *et al.*, 2012).

Algumas das vantagens de utilizar modelos de EESA incluem (Idri; Amazal; Abran, 2015):

Tendem a produzir estimativas aceitáveis, muitas vezes superando outros modelos de estimativa, conforme indicado em estudos (Idri; Amazal; Abran, 2015); Facilitam a modelagem de relações complexas entre o esforço e os atributos do software, podendo ser aplicados em uma fase inicial de um projeto de software; Diferentemente de outras técnicas que utilizam Redes Neurais Artificiais, do Inglês *Artificial Neural Networks* (ANN), que são vistas como uma caixa-preta, as técnicas baseadas em analogia são facilmente compreendidas pelos usuários, pois imitam o raciocínio humano por analogia. A EESA pode ser aplicada tanto em modelos de desenvolvimento tradicionais (ex. Processo Unificado) quanto em modelos de desenvolvimento ágeis (ex. Scrum), já que ambos se baseiam em experiências anteriores da equipe para planejar novos projetos de software. No entanto, o processo de estimativa por analogia, especialmente em metodologias ágeis, geralmente é realizado por humanos mediante discussões entre os membros da equipe, sem uma utilização efetiva de dados históricos de projetos. Além disso, devido à natureza ágil, esses dados muitas vezes não são registrados de maneira adequada (Cohn, 2005).

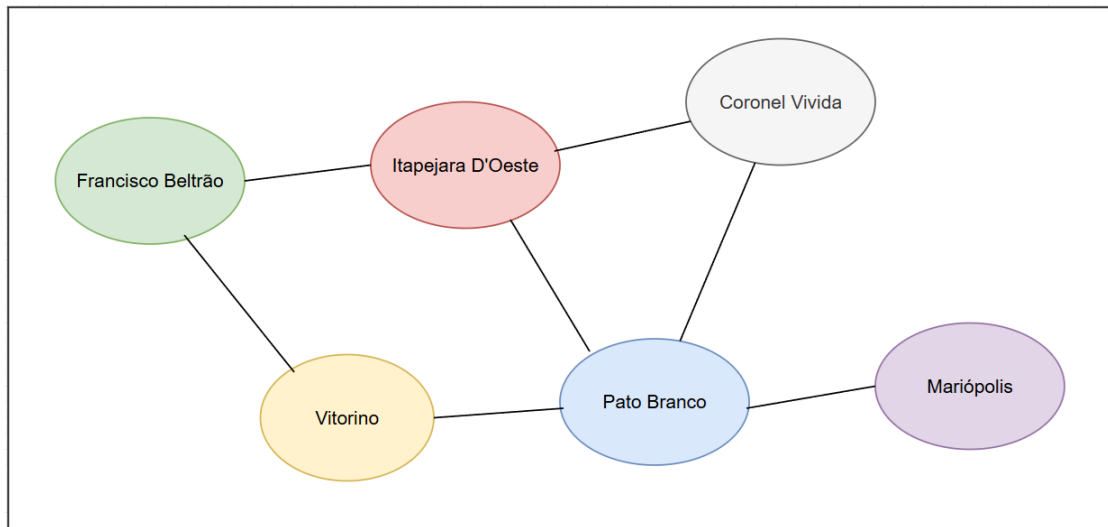
A metodologia de desenvolvimento ágil de software, amplamente adotada por empresas de desenvolvimento de software de todos os tamanhos (pequenas, médias ou grandes), é uma abordagem flexível que permite entregas mais rápidas aos *stakeholders* por meio de um processo simplificado e com menos geração de documentação, em comparação com processos de desenvolvimento de software tradicionais. Nesse modelo, a mensuração de tamanho e a geração de estimativas de tempo e custos geralmente fica sob a responsabilidade da equipe de desenvolvimento, tornando-se um processo bastante subjetivo (Cohn, 2005).

2.3 Grafos

Segundo (Harary, 1969), um grafo é definido como uma estrutura matemática que consiste em um conjunto de vértices ou nós e um conjunto de arestas ou arcos que conectam esses vértices. Os vértices são as unidades fundamentais de um grafo. As arestas indicam relacionamentos ou interações. O autor descreve os grafos como uma forma de modelar relações entre pares objetos de uma determinada coleção. No contexto da teoria dos grafos, um grafo é representado como um par $G = (V, E)$, onde V é o conjunto de vértices e E é o conjunto de arestas conectando esses vértices. Os grafos podem ser classificados de acordo com propriedades ou características específicas, como os simples, direcionados (dígrafos), mistos, ponderados (valorados) e conexos. Na Figura 1 é apresentado um grafo, onde as cidades são representadas como nós.

Adjacência em grafos, conforme explicado por (Harary, 1969), refere-se à relação entre dois vértices conectados por uma aresta. Dois vértices são considerados adjacentes se houver uma aresta ligando-os diretamente. Esse conceito é fundamental na teoria dos grafos, pois define a conectividade entre os vértices, ajudando a determinar as relações e interações entre os diferentes elementos representados. A adjacência pode representar vários tipos de relaciona-

Figura 1 – Exemplo de cidades e suas conexões representadas por um grafo.

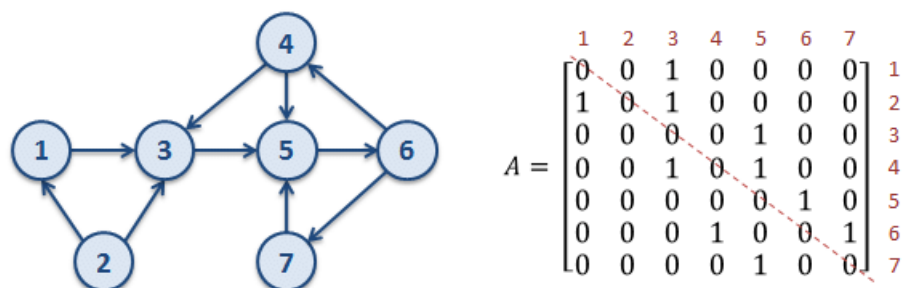


Fonte: Autoria própria (2025).

mentos, como conexões sociais em redes sociais (Wilson *et al.*, 2009), proximidade espacial em grafos geográficos (Backstrom; Sun; Marlow, 2010) e dependências entre tarefas em grafos de gerenciamento de projetos (Heimann *et al.*, 1997). Compreender a adjacência é fundamental para analisar a estrutura e as propriedades dos grafos, influenciando algoritmos e processos usados na teoria dos grafos, como a descoberta de caminhos. Portanto, listas de adjacência ou matrizes de adjacência são uma forma comum de representar essas relações, facilitando a identificação de vértices adjacentes e a análise de padrões de conectividade.

A Figura 2 ilustra um exemplo de grafo e sua correspondente matriz de adjacência. No grafo, cada nó é representado por um círculo numerado de 1 a 7, e as arestas direcionadas indicam a conexão entre os nós. A matriz de adjacência A , mostrada ao lado direito do grafo, é uma matriz quadrada de ordem 7, onde a posição A_{ij} contém o valor 1 se houver uma aresta do nó i para o nó j , e 0 caso contrário.

Figura 2 – Exemplo de um grafo e sua matriz de adjacência



(Rodrigues, 2018)

2.4 Redes Neurais Artificiais (ANNs)

As redes neurais artificiais são sistemas computacionais inspirados na estrutura e funcionamento do cérebro humano. Elas consistem em unidades interconectadas chamadas neurônios artificiais (Haykin, 2001). Desde a criação do Perceptron por Frank Rosenblatt nos anos 1950, as redes neurais têm evoluído significativamente, culminando na popularização das redes neurais profundas na última década.

Redes neurais profundas (Deep Neural Networks, DNNs) são redes neurais com múltiplas camadas ocultas que aprendem representações hierárquicas dos dados: camadas iniciais capturam características simples e, à medida que a profundidade aumenta, emergem atributos mais abstratos e específicos da tarefa. Essa profundidade permite compor funções complexas a partir de funções simples e tem sido central para avanços recentes em visão computacional, processamento de linguagem natural e reconhecimento de fala (LeCun; Bengio; Hinton, 2015)(Goodfellow; Bengio; Courville, 2016).

Um neurônio artificial é a unidade básica das redes neurais. Ele recebe entradas, que são ponderadas, processadas por uma função de ativação, e produzem uma saída. As redes neurais são compostas por camadas: a camada de entrada, que recebe os dados brutos; as camadas escondidas, que processam os dados; e a camada de saída, que produz o resultado final (Haykin, 2001).

No processo de propagação direta (do inglês, *forward propagation*), os dados são transmitidos da camada de entrada para a camada de saída por meio das camadas intermediárias. Durante o treinamento, o algoritmo de retropropagação (do inglês *backpropagation*) ajusta os pesos dos neurônios com base no erro entre a saída esperada e a saída real, permitindo que a rede aprenda e melhore seu desempenho (Goodfellow; Bengio; Courville, 2016).

Alguns tipos de Redes Neurais:

- **Perceptron Multicamadas (do inglês, MLP):** As redes neurais *feedforward* mais simples, usadas em diversas aplicações (Popescu *et al.*, 2009).
- **Redes Neurais Convolucionais (do inglês, CNN):** Essenciais para tarefas de processamento de imagens, como reconhecimento de objetos. A convolução é uma operação fundamental em CNNs, em visão computacional, por exemplo, aplica-se um filtro (kernel) sobre a entrada para extrair características locais, como bordas e texturas, reduzindo a complexidade computacional e capturando padrões importantes. (O'shea; Nash, 2015).
- **Redes Neurais Recorrentes (do inglês, RNN):** Utilizadas para dados sequenciais, incluindo séries temporais e PLN (Salehinejad *et al.*, 2017).

- **Redes Neurais em Grafos (do inglês, GNN):** Específicas para dados estruturados em grafos, permitindo o aprendizado em dados que possuem relações complexas, como redes sociais, moléculas e sistemas de recomendação (Wu *et al.*, 2020).
- **Redes Neurais com Atenção e Transformers:** Apresentaram avanços significativos no campo de PLN, com aplicações em tradução automática e geração de texto (Xiao; Zhu, 2023).
- **Redes Convolucionais em Grafos (do inglês, GCN):** são uma classe de redes neurais que operam diretamente em grafos, generalizando convoluções de redes neurais para estruturas de grafos. Ela captura informações estruturais e relacionais entre os nós do grafo, permitindo a aprendizagem de representações eficientes para tarefas como classificação de nós e arestas (Zhang *et al.*, 2019).

A seguir é apresentado mais detalhes sobre as GCNs, que são aplicadas posteriormente nos métodos dessa pesquisa.

2.5 Redes Convolucionais em Grafos (GCNs)

As GCNs, ou ainda, Redes Neurais Convolucionais Baseadas em Grafos, representam um avanço significativo no campo do aprendizado de máquina, especificamente projetadas para lidar com dados que possuem uma estrutura de grafo. Ao contrário das redes neurais tradicionais, que operam em dados tabulares ou sequenciais, as GCNs são capazes de capturar dependências complexas e relacionamentos presentes em dados representados como nós e arestas (Zhang *et al.*, 2019).

Como explicado anteriormente, grafos são compostos por nós (ou vértices) e arestas que conectam esses nós, em uma GCN, cada nó possui uma representação inicial (ou vetor de características) que é atualizado ao longo das camadas da rede. As GCNs aplicam operações convolucionais sobre os nós do grafo, considerando as conexões entre eles, isso permite a agregação de informações dos nós vizinhos, capturando a estrutura local do grafo. Durante a propagação, cada nó atualiza sua representação, agregando informações de seus vizinhos, essa operação é análoga à convolução em redes convolucionais tradicionais, mas adaptada para a topologia de grafos.

As GCNs têm sido aplicadas com sucesso em diversas áreas como análise de redes sociais para detectar comunidades, influenciadores e prever conexões futuras. Na química e biologia com previsão de propriedades de moléculas e análise de redes biológicas. E também em sistemas de recomendação melhorando a recomendação de itens ao capturar relações complexas entre usuários e produtos.

2.6 Operação de Convolução em Grafos

A operação de convolução em grafos é uma extensão das convoluções tradicionais, aplicadas em dados não estruturados como imagens, para dados estruturados em forma de grafos. Grafos são compostos por nós (vértices) e arestas (conexões), e a convolução em grafos visa agregar informações de um nó e seus vizinhos para criar representações ricas dos nós.

2.6.1 Definição e Formalização

De acordo com (Kipf; Welling, 2017), em uma GCN, a convolução em um nó v_i é definida como a agregação das características do nó v_i e dos nós vizinhos $N(v_i)$. A operação pode ser formalizada como:

$$H^{(l+1)} = \sigma \left(\tilde{A} H^{(l)} W^{(l)} \right)$$

onde:

- $H^{(l)}$ é a matriz de características dos nós na camada l .
- $\tilde{A}$ é a matriz de adjacência normalizada do grafo.
- $W^{(l)}$ é a matriz de pesos da camada l .
- σ é a função de ativação, como ReLU¹.

A matriz de adjacência normalizada $\tilde{A}$ é calculada como:

$$\tilde{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$$

onde D é a matriz diagonal de graus², e A é a matriz de adjacência do grafo.

2.6.2 Exemplo Prático

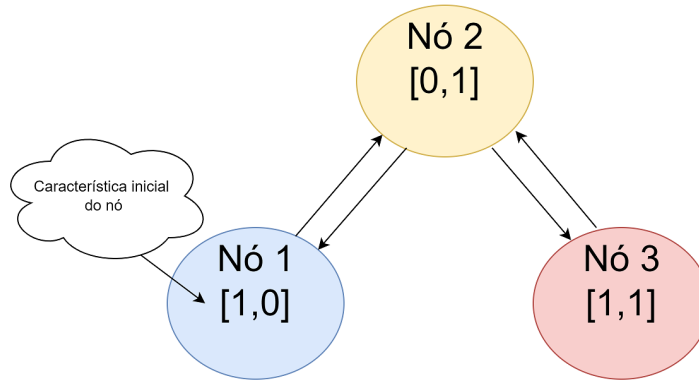
Considere a Figura 3, um grafo simples com 3 nós e as seguintes conexões:

- Nó 1 está conectado ao nó 2.
- Nó 2 está conectado ao nó 1 e ao nó 3.

¹ A ReLU (*Rectified Linear Unit*) é uma função de ativação amplamente utilizada em redes neurais que retorna o valor de entrada se for positivo, e zero se for negativo. Sua fórmula é $\text{ReLU}(x) = \max(0, x)$, ajuda a introduzir não-linearidade ao modelo (Agarap, 2018).

² O grau de um vértice em um grafo simples é o número de arestas incidentes a esse vértice, isto é, o número de vizinhos aos quais ele está conectado (Diestel, 2017).

Figura 3 – Exemplo de grafo com características iniciais dos nós.



Fonte: Autoria própria (2025)

- Nó 3 está conectado ao nó 2.

A matriz de adjacência A e a matriz diagonal de graus D são:

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad D = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

A matriz de adjacência normalizada³ $\tilde{A}$ é:

$$\tilde{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ 0 & \frac{1}{\sqrt{2}} & 0 \end{pmatrix}$$

A matriz $H^{(0)}$ de características iniciais dos nós:

$$H^{(0)} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$$

Se a matriz $W^{(0)}$ iniciada com pesos aleatórios for:

³ Essa transformação corresponde à *normalização simétrica da matriz de adjacência*, que está diretamente relacionada ao laplaciano normalizado do grafo. Ela é utilizada para equilibrar a influência dos vizinhos na etapa de agregação, evitando que nós de alto grau dominem a propagação de mensagens e tornando a operação de convolução em grafos mais estável ao longo das camadas da GCN (Kipf; Welling, 2017).

$$W^{(0)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

A nova matriz de características dos nós $H^{(1)}$ após a primeira camada de convolução é:

$$H^{(1)} = \sigma(\tilde{A}H^{(0)}W^{(0)})$$

exemplificando os cálculos:

1. Multiplicação de $\tilde{A}$ e $H^{(0)}$:

$$\tilde{A}H^{(0)} = \begin{pmatrix} 0 & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ 0 & \frac{1}{\sqrt{2}} & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 0 & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 1 & 1 \end{pmatrix}$$

2. Multiplicação do resultado pela matriz de pesos $W^{(0)}$:

$$H^{(1)} = \sigma \left(\begin{pmatrix} 0 & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \right) = \sigma \left(\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 1 & 1 \\ 2 & 2 \end{pmatrix} \right)$$

Supondo que σ é a função ReLU, é obtido:

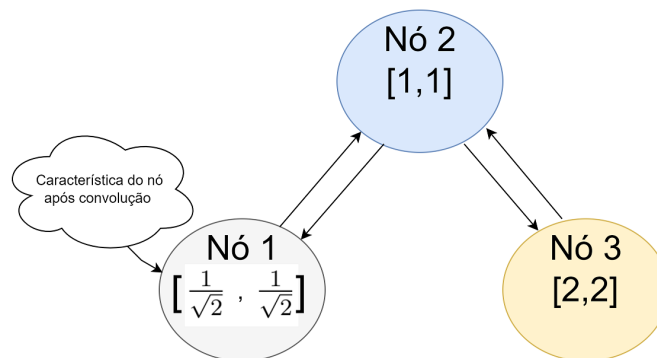
$$H^{(1)} = \begin{pmatrix} \text{ReLU}\left(\frac{1}{\sqrt{2}}\right) & \text{ReLU}\left(\frac{1}{\sqrt{2}}\right) \\ \text{ReLU}(1) & \text{ReLU}(1) \\ \text{ReLU}(2) & \text{ReLU}(2) \end{pmatrix}$$

$$H^{(1)} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 1 & 1 \\ 2 & 2 \end{pmatrix}$$

Este processo mostra como as características dos nós são atualizadas em cada camada de convolução, combinando as características dos nós vizinhos e aplicando uma transformação linear seguida por uma função de ativação. Na Figura 4 é ilustrada a representação do grafo após a operação de convolução descrita anteriormente.

Para compreender o funcionamento do modelo de representação textual utilizado neste trabalho, que é empregado para inicializar a matriz de características mencionada anteriormente, é importante considerar os conceitos a seguir.

Figura 4 – Exemplo de grafo com representação de suas características atualizadas após operação de convolução.



Fonte: Autoria própria (2025)

2.7 Representação Textual

Para que um computador possa classificar textos e extrair conhecimento e contexto, é necessário converter esses textos em um formato que a máquina possa interpretar (Naseem *et al.*, 2021). Para isso, utilizam-se modelos de representação textual que transformam palavras em vetores (Liu; Lin; Sun, 2020).

Existem várias formas de representação das características textuais na literatura. Entre as mais destacadas estão os modelos clássicos, que incluem representações categóricas e ponderadas (ex. bag-of-words), e os modelos de *embeddings*.

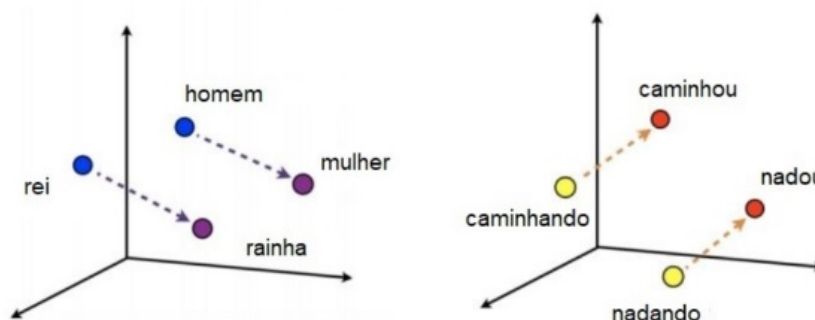
Uma abordagem mais recente e avançada é a representação textual por grafos. Nesta técnica, os textos são transformados em grafos onde as palavras ou sentenças são representadas como nós, e as relações entre elas, como arestas. Essa estrutura permite capturar dependências sintáticas e semânticas complexas que os métodos tradicionais podem não conseguir modelar de maneira eficaz. A representação por grafos tem mostrado ser eficaz em várias tarefas de PLN, como classificação de texto, geração de texto e análise de sentimentos (Yao; Mao; Luo, 2018).

A seguir são apresentadas algumas técnicas de representação textual que vem sendo utilizadas em pesquisas sobre PLN.

2.7.1 Word Embeddings

Um *word embedding*, ou apenas *embedding*, é uma representação gerada a partir de um modelo pré-treinado (ex. GloVe, BERT, *Transformer* Pré-treinado Generativo, do inglês *Generative Pre-trained Transformer* (GPT)), o qual utiliza métodos de redes neurais para extrair características textuais (Bengio; Courville; Vincent, 2013a). Palavras são mapeadas de maneira

Figura 5 – Exemplos de plot 3D de word vectors.



Fonte: Adaptado de Fonseca (2021)

tal, que palavras próximas no espaço vetorial tenham significados semelhantes (Jurafsky; Martin, 2000). No campo do PLN, técnicas não supervisionadas de representação de texto, como *embeddings*, têm substituído métodos clássicos devido à sua capacidade de descobrir de forma autônoma representações textuais (Naseem *et al.*, 2021).

Na representação por *embeddings*, cada palavra é tipicamente representada por um vetor de valores reais com centenas ou milhares de dimensões (Goldberg, 2017). A Figura 5 ilustra um exemplo de representação vetorial de *embeddings* para formas comparativas e superlativas de adjetivos, bem como para tempos verbais.

A representação geométrica entre os *embeddings* permite capturar relações semânticas, como a existente entre um verbo no infinitivo e suas flexões (Mikolov *et al.*, 2013). Assim, torna-se possível expressar a importância semântica de uma palavra em formato numérico (Zhang *et al.*, 2015).

Como os modelos de *embeddings* representam palavras por meio de vetores de valores reais, é possível posicionar uma palavra em um ponto em um espaço N-dimensional. Isso permite calcular a distância entre dois pontos, representando a distância entre duas palavras, ou ainda, sua similaridade (Goldberg, 2017), conforme ilustrado na Figura 5.

2.7.2 Modelos de Representação Pré-treinados

O pré-treinamento de modelos de linguagem provou ser altamente eficaz no aprendizado de representações universais de linguagem a partir de dados não rotulados em grande escala (Sun *et al.*, 2019).

Os modelos de *embeddings* pré-treinados são bem genéricos em sua representação e funcionam como uma base robusta de conhecimento para aplicações com pouca quantidade de dados ou dados esparsos (Jurafsky; Martin, 2019; Garg; Sharma; Liang, 2020). Estudos recentes (Radford *et al.*, 2018; Radford *et al.*, 2019; Brown *et al.*, 2020) demonstram que seu uso possui a capacidade de aumentar a performance de modelos de PLN, melhorar a qualidade das representações, e ainda, permitir uma rápida especialização para uma tarefa.

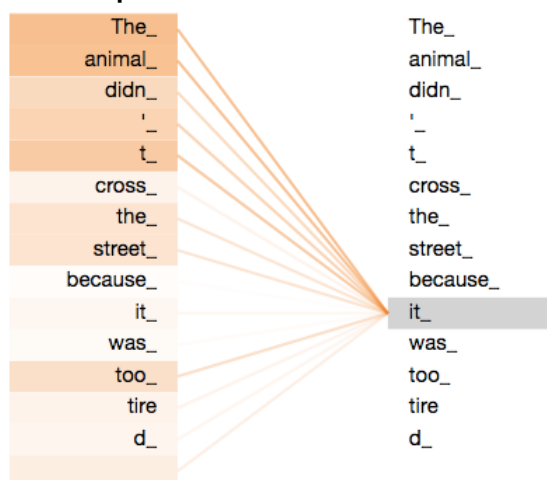
Para construir um modelo pré-treinado, são utilizados grandes conjuntos de dados em linguagem natural, envolvendo conteúdos de diversas áreas, que são pré-processados e preparados para serem dados como entrada no aprendizado do modelo. Após a etapa de pré-treino, esses modelos podem ser utilizados para transferir conhecimento para outras tarefas (Jurafsky; Martin, 2019).

Dentre os modelos de *embeddings* pré-treinados mais utilizados atualmente, destacam-se o BERT (Devlin *et al.*, 2018) e a família GPT (Radford *et al.*, 2018). O motivo principal é que ambos se utilizam do mecanismo de *Transformers* (Vaswani *et al.*, 2017) (ou transformadores, em português).

Os *Transformers* são um tipo de arquitetura de rede neural, que reduzem a necessidade da computação sequencial de sentenças textuais a partir de seus vários módulos de auto-atenção, que permitem a identificação mais precisa do contexto específico de cada palavra, com base nas palavras que compõem o texto. Ele pondera diferencialmente o significado de cada parte dos dados de entrada e processa toda a entrada de uma só vez. Sua arquitetura interna é composta por dois mecanismos, um codificador e um decodificador (respectivamente *encoder* e *decoder*, do inglês)(Vaswani *et al.*, 2017).

Na Figura 6 apresenta-se um exemplo do uso do mecanismo de auto-atenção no processamento da frase em inglês “*The animal didn’t cross the street because it was too tired*”. Nessa sentença, o pronome neutro “*it*” (que, diferentemente de “ele/ela” em português, não possui marca explícita de gênero) pode, a princípio, referir-se tanto a “*the street*” quanto a “*the animal*”, configurando uma ambiguidade de referência. O mecanismo de auto-atenção, ao processar o token “*it*”, aprende a atribuir maior peso às conexões com o token “*animal*” na mesma frase, resolvendo essa ambiguidade. Ainda na Figura 6, os tokens à esquerda destacados em tons mais intensos de laranja indicam maior peso de atenção em relação ao token “*it*” (destacado à direita), conforme a ilustração original de Alammar (2018).

Figura 6 – Exemplo de uso do mecanismo de auto-atenção.



Fonte: Alammar (2018)

2.7.2.1 BERT

O *Bidirectional Encoder Representations from Transformers* (BERT) é um modelo de linguagem pré-treinado publicado pelos pesquisadores Devlin *et al.* (2018), do departamento de *AI Language* da Google. Segundo os autores, a inovação técnica do BERT é aplicar o treinamento bidirecional do *Transformer* (Vaswani *et al.*, 2017), um modelo de auto-atenção (do inglês, *self-attention*) popular aos modelos de linguagem pré-treinados contextualizados. Os resultados do artigo (Devlin *et al.*, 2018) mostram que um modelo de linguagem treinado bidirecionalmente pode ter um senso mais profundo de contexto e fluxo de linguagem do que modelos de linguagem unidirecionais.

Deste modo, o mecanismo *Transformer* permite analisar o contexto de cada palavra em um texto de forma individual, o que permite verificar se cada palavra já foi utilizada anteriormente em um mesmo contexto. Com isso, o método permite aprender relações contextuais entre palavras (ou sub-palavras) em um texto. Assim, o BERT é um modelo de linguagem em larga escala que consiste em várias camadas de blocos do tipo *Transformer*.

Isso se explica, porque o BERT tem se mostrado eficiente em diversas tarefas de PLN, pois utiliza o mecanismo codificador do *Transformer*, o qual, ao contrário dos modelos direcionais, que lêem a entrada de texto sequencialmente (da esquerda para a direita ou da direita para a esquerda), lêem toda a sequência de palavras de uma só vez e em ambas as direções, permitindo que o modelo aprenda o contexto de uma palavra com base em todos os seus arredores (esquerda e direita da palavra) (Devlin *et al.*, 2018).

Além disso, o BERT usa duas estratégias de treinamento: o *Masked Language Modeling* (MLM), na qual algumas das palavras em cada sequência são substituídas por um *token* de máscara e, em seguida, o modelo tenta prever o valor original delas com base no contexto fornecido pelas outras palavras (não mascaradas) na sequência, e o *Next Sentence Prediction* (NSP), na qual o modelo recebe pares de sentenças como entrada e aprende a prever se a segunda sentença do par é a sentença subsequente no documento original. Ao treinar o modelo BERT, os preditores do MLM e do NSP são treinados juntos, com o objetivo de minimizar a função de perda combinada das duas estratégias (Devlin *et al.*, 2018).

2.7.3 Representação Textual por Grafos

Uma abordagem amplamente estudada para a representação textual é o modelo baseado em grafos. Nesta técnica, os textos são convertidos em grafos em que palavras, sentenças ou documentos são representados como nós, e as relações entre eles são representadas como arestas. Esse método é especialmente útil para capturar a estrutura sintática e semântica presente em textos.

A construção do grafo pode ser realizada de diferentes maneiras. Em modelos baseados em coocorrência, é comum conectar palavras que aparecem próximas em uma janela deslizando

de tamanho fixo, de modo que pares de palavras que coocorrem com maior frequência nessa janela recebam pesos mais altos. Modelos de classificação de textos baseados em grafos, como o TextGCN, constroem um grafo em nível de corpus com nós de documentos e palavras, em que as arestas documento–palavra são ponderadas por TF–IDF e as arestas palavra–palavra são ponderadas por *Positive Pointwise Mutual Information* (PPMI) calculada a partir da coocorrência de palavras em janelas deslizantes ao longo dos documentos (Yao; Mao; Luo, 2018). Alternativamente, podem ser utilizadas dependências gramaticais para definir as arestas entre palavras em uma sentença, ou ainda relações de similaridade entre sentenças e documentos.

Cada nó no grafo é associado a um vetor de características, que pode variar desde representações esparsas (por exemplo, vetores one-hot ou a matriz identidade, como em (Yao; Mao; Luo, 2018)) até *embeddings* densos pré-treinados para palavras, sentenças ou documentos. Em seguida, técnicas de aprendizado em grafos, como as GCNs, são aplicadas para propagar informação entre os nós e extrair características relevantes para tarefas específicas de PLN.

A principal vantagem dessa abordagem é a capacidade de modelar explicitamente as relações entre palavras, sentenças e documentos, o que pode melhorar o desempenho em tarefas como classificação de texto e sumarização. Além disso, a representação gráfica permite integrar informações contextuais e estruturais de maneira mais natural e expressiva do que representações puramente baseadas em *embeddings* independentes.

2.8 Classificação Textual por GCNs

Modelos baseados em GCNs têm sido investigados como alternativas competitivas para tarefas de classificação textual, pois permitem representar textos como grafos e explorar explicitamente relações entre seus componentes. Ao representar textos como grafos, em que palavras, sentenças ou documentos são nós conectados por arestas que codificam relações semânticas ou sintáticas, as GCNs podem explorar essas interconexões para construir representações úteis para a classificação (Kipf; Welling, 2017; Yao; Mao; Luo, 2018).

Segundo os autores (Yao; Mao; Luo, 2018), o processo de classificação textual com GCNs envolve várias etapas:

- **Construção do grafo:** inicialmente, os textos são transformados em grafos. Isso pode ser feito utilizando técnicas de coocorrência, dependências gramaticais, ou outras métricas que definem a relação entre palavras, sentenças ou documentos.
- **Inicialização dos nós:** cada nó no grafo é associado a um vetor que representa a palavra, sentença ou documento correspondente. Esses vetores podem ser *embeddings* pré-treinados ou aprendidos durante o treinamento do modelo.

- **Propagação e agregação:** as GCNs aplicam operações de convolução sobre os nós do grafo, permitindo que cada nó agregue informações de seus vizinhos. Esse processo é análogo ao das CNNs em imagens, porém adaptado à topologia de grafos.
- **Classificação:** após uma ou mais camadas de convolução e agregação, as representações finais dos nós ou do grafo como um todo são utilizadas em uma camada de saída (por exemplo, uma *softmax*) para produzir as categorias preditas.

Resultados empíricos em tarefas como análise de sentimentos, detecção de tópicos e categorização de documentos indicam que modelos baseados em GCNs podem alcançar desempenho comparável ou superior a abordagens tradicionais que tratam o texto apenas como sequências ou *bags-of-words* (Kipf; Welling, 2017; Yao; Mao; Luo, 2018). Em muitos desses modelos, a tarefa de classificação textual é formulada em um cenário transdutivo, no qual o grafo incorpora simultaneamente exemplos rotulados e não rotulados. A Seção 2.9 apresenta os fundamentos desse paradigma, que serve de base para o modelo híbrido entre GCN e BERT discutido posteriormente neste trabalho.

2.9 Aprendizado Transdutivo

O aprendizado transdutivo é um paradigma de aprendizado de máquina que difere fundamentalmente do aprendizado indutivo tradicional. Enquanto o aprendizado indutivo busca aprender uma função geral de mapeamento dos dados de entrada para as saídas, que possa ser aplicada a novos exemplos não vistos, o aprendizado transdutivo concentra-se especificamente em fazer previsões para um conjunto fixo de exemplos não rotulados conhecidos durante o treinamento (Vapnik, 1998).

2.9.1 Fundamentos Teóricos

O conceito de aprendizado transdutivo foi formalizado por Vapnik (1998) na teoria do aprendizado estatístico. Segundo Vapnik, quando o objetivo é resolver um problema específico (prever rótulos para um conjunto particular de exemplos não rotulados), não é necessário resolver o problema mais geral de induzir uma função de classificação completa. O aprendizado transdutivo explora diretamente a estrutura dos dados de teste durante o treinamento, potencialmente alcançando melhor desempenho com menos exemplos rotulados (Gammerman; Vovk; Vapnik, 2013).

Formalmente, dado um conjunto de treinamento rotulado $\mathcal{L} = \{(x_1, y_1), \dots, (x_l, y_l)\}$ e um conjunto de teste não rotulado $\mathcal{U} = \{x_{l+1}, \dots, x_{l+u}\}$, o aprendizado transdutivo busca prever os rótulos $\{y_{l+1}, \dots, y_{l+u}\}$ para os exemplos em $\mathcal{U}$, utilizando informações de ambos os conjuntos $\mathcal{L}$ e $\mathcal{U}$ simultaneamente (Chapelle; Schölkopf; Zien, 2006).

2.9.2 Diferenças entre Aprendizado Indutivo e Transdutivo

As principais diferenças entre esses paradigmas incluem:

- **Objetivo:** O aprendizado indutivo busca generalizar para qualquer exemplo futuro, enquanto o transdutivo foca apenas nos exemplos específicos do conjunto de testes conhecido (Joachims, 1999).
- **Uso de dados não rotulados:** No aprendizado transdutivo, os dados de teste não rotulados são utilizados durante o treinamento para melhorar as previsões, explorando a estrutura intrínseca dos dados. No aprendizado indutivo supervisionado, apenas dados rotulados são utilizados no treinamento (Chapelle; Schölkopf; Zien, 2006).
- **Complexidade:** O aprendizado transdutivo pode ser teoricamente mais eficiente que o indutivo quando o número de exemplos de teste é fixo e conhecido, pois evita a necessidade de aprender uma função de mapeamento completa (Vapnik, 1998).
- **Aplicabilidade:** Modelos indutivos podem ser aplicados a novos dados a qualquer momento, enquanto modelos transdutivos precisam ser retreinados ao receber novos exemplos não rotulados (Gammerman; Vovk; Vapnik, 2013).

2.9.3 Aprendizado Semi-Supervisionado e Transdutivo

O aprendizado transdutivo está intimamente relacionado ao aprendizado semi-supervisionado, que também utiliza dados rotulados e não rotulados (Zhu, 2005). No entanto, existe uma distinção importante: o aprendizado semi-supervisionado indutivo aprende um modelo que pode ser aplicado a novos dados, enquanto o aprendizado transdutivo faz previsões apenas para o conjunto específico de exemplos não rotulados fornecidos durante o treinamento (Chapelle; Schölkopf; Zien, 2006).

Segundo Zhu (2005), o aprendizado semi-supervisionado baseia-se em suposições sobre a estrutura dos dados, incluindo:

- **Suposição de suavidade (*smoothness assumption*):** Pontos próximos no espaço de características tendem a ter o mesmo rótulo.
- **Suposição de agrupamento (*cluster assumption*):** Os dados formam agrupamentos discretos, e pontos no mesmo agrupamento tendem a compartilhar o mesmo rótulo.
- **Suposição de variedade (*manifold assumption*):** Os dados de alta dimensão residem aproximadamente em uma variedade de dimensão menor, e a estrutura dessa variedade pode ser explorada para melhorar o aprendizado (Belkin; Niyogi; Sindhwani, 2006).

2.9.4 Propagação de Rótulos em Grafos

Uma das técnicas mais populares no aprendizado transdutivo é a propagação de rótulos em grafos, onde os dados são representados como um grafo com nós (exemplos) e arestas (similaridades) (Zhu; Ghahramani, 2002). O algoritmo propaga rótulos dos nós rotulados para os nós não rotulados por meio das arestas do grafo, baseando-se na suposição de que nós conectados por arestas com pesos altos tendem a ter o mesmo rótulo (Zhou *et al.*, 2003).

Zhu e Ghahramani (2002) propuseram um algoritmo de propagação de rótulos que itera até a convergência, atualizando os rótulos dos nós não rotulados como uma média ponderada dos rótulos de seus vizinhos. Este método é particularmente eficaz quando a estrutura do grafo reflete bem a similaridade semântica entre os exemplos.

Variantes mais sofisticadas incluem o método de campos aleatórios de Markov (*Markov Random Fields*) (Zhu; Ghahramani; Lafferty, 2003) e métodos baseados em difusão de calor no grafo (Kondor; Lafferty, 2002), que consideram não apenas vizinhos diretos, mas também a estrutura global do grafo para propagar informações de rótulos.

2.9.5 Aplicações em Processamento de Linguagem Natural

No contexto de PLN, o aprendizado transdutivo tem sido aplicado com sucesso em tarefas como classificação de documentos, análise de sentimentos e categorização de tópicos (Joachims, 1999). A representação de textos como grafos, onde documentos e palavras são nós conectados por relações semânticas, permite que algoritmos transdutivos explorem a estrutura intrínseca do corpus para melhorar as previsões (Yao; Mao; Luo, 2018).

2.9.6 Vantagens e Limitações

As principais vantagens do aprendizado transdutivo incluem (Gammerman; Vovk; Vapnik, 2013):

- **Eficiência com poucos dados rotulados:** Pode alcançar melhor desempenho que métodos indutivos quando há escassez de exemplos rotulados, explorando a estrutura dos dados não rotulados.
- **Exploração da estrutura dos dados:** Utiliza informações sobre a distribuição dos dados de teste para melhorar as previsões.
- **Garantias teóricas de eficiência:** A formulação transdutiva é apoiada por resultados formais da teoria estatística da aprendizagem, que demonstram, em cenários específicos, maior eficiência em relação ao aprendizado indutivo (Vapnik, 1998).

No entanto, existem limitações importantes (Chapelle; Schölkopf; Zien, 2006):

- **Falta de generalização:** Modelos transdutivos não podem ser diretamente aplicados a novos exemplos fora do conjunto de teste original, sendo necessário retreinar o modelo.
- **Custo computacional:** Algoritmos transdutivos podem ser computacionalmente custosos, especialmente para grandes conjuntos de dados, pois precisam considerar todos os exemplos simultaneamente.
- **Sensibilidade à estrutura do grafo:** O desempenho depende fortemente de como o grafo é construído e da qualidade das medidas de similaridade utilizadas (Zhu, 2005).

O aprendizado transdutivo constitui uma abordagem poderosa quando o conjunto de exemplos a serem classificados é conhecido antecipadamente e há interesse em explorar a estrutura desses dados para melhorar as previsões. Esta característica torna-o particularmente adequado para tarefas de classificação de documentos em corpora fechados, como é o caso da arquitetura BertGCN apresentada a seguir.

2.10 BertGCN

O BertGCN é uma abordagem inovadora para a classificação transdutiva de textos que combina pré-treinamento em larga escala e aprendizado transdutivo (Lin *et al.*, 2022). O modelo constrói um grafo heterogêneo sobre o conjunto de dados e representa os documentos como nós usando representações do BERT. Treinando conjuntamente os módulos BERT e GCN, o BertGCN tira proveito do pré-treinamento em larga escala, que aproveita uma grande quantidade de dados brutos, e do aprendizado transdutivo, que aprende representações para os dados de treinamento e de teste não rotulados, propagando a influência dos rótulos mediante convolução de grafos (Lin *et al.*, 2022).

Para construir o grafo, são definidos nós de palavras e documentos, com arestas baseadas em TF-IDF e PPMI (Yao; Mao; Luo, 2018). As representações dos nós de documentos são inicializadas com *embeddings* do BERT e iterativamente atualizadas com GCNs. As representações finais dos documentos são então usadas para previsões com uma camada softmax. Este processo permite que o modelo combine as vantagens dos modelos pré-treinados e dos grafos, resultando em um desempenho superior na classificação de textos (Lin *et al.*, 2022).

No modelo, a informação mútua pontual positiva, do inglês (*Positive Pointwise Mutual Information* (PPMI)), é utilizada para definir as arestas entre nós de palavras no grafo. Primeiro, calcula-se a matriz de coocorrência das palavras, que registra quantas vezes cada par de palavras aparece no mesmo documento. A partir dessa matriz, obtém-se a matriz PPMI, que ajusta os valores negativos de PMI para zero, garantindo que somente associações positivas e significativas sejam consideradas. As arestas entre os nós de palavras são então criadas com base nos valores da PPMI, onde valores maiores que um limiar predefinido indicam uma conexão

forte e resultam na criação de uma aresta no grafo. Esse processo ajuda a capturar as relações semânticas positivas entre palavras, enriquecendo a representação do grafo.

Term Frequency-Inverse Document Frequency (TF-IDF) é utilizado no modelo para definir as arestas entre nós de documentos e nós de palavras. Para cada documento no conjunto de dados, calcula-se a matriz TF-IDF, que mede a importância de cada palavra em relação a um documento específico, ponderada pela frequência inversa dessa palavra em todo o corpus. As arestas são criadas entre os nós de documentos e os nós de palavras com base nos valores de TF-IDF: se o valor de TF-IDF de uma palavra em um documento for maior que um limiar estabelecido, uma aresta é criada entre o nó do documento e o nó da palavra. Isso assegura que palavras representativas de um documento tenham conexões fortes, ajudando a capturar a relevância local das palavras no contexto dos documentos.

Os experimentos realizados em vários *benchmarks*⁴ de classificação de texto mostram que o BertGCN alcança desempenho notável, superando modelos baseados apenas em BERT ou GCN, especialmente em conjuntos de dados com documentos longos. Esta abordagem demonstra a eficácia do BertGCN em utilizar dados rotulados e não rotulados, estabelecendo um novo padrão de desempenho em diversas tarefas de classificação de texto (Lin *et al.*, 2022).

2.11 Métricas e Medidas Estatísticas para Análise Exploratória de Dados

A análise exploratória de dados (EDA, do inglês *Exploratory Data Analysis*) é uma etapa fundamental em qualquer projeto de ciência de dados ou aprendizado de máquina, pois permite compreender as características, padrões e peculiaridades do conjunto de dados antes da aplicação de modelos preditivos (Tukey, 1977). Segundo o autor, a EDA enfatiza a visualização e a aplicação de técnicas estatísticas descritivas para descobrir estruturas subjacentes, detectar anomalias, testar hipóteses e verificar suposições. Neste contexto, diversas métricas e medidas estatísticas são empregadas para caracterizar a distribuição dos dados, avaliar sua qualidade e identificar fatores que possam influenciar o desempenho de modelos de aprendizado de máquina.

2.11.1 Medidas de Tendência Central

As medidas de tendência central são utilizadas para descrever o valor típico ou central de uma distribuição de dados (Montgomery; Runger, 2018). As principais medidas incluem:

- **Média aritmética (μ):** Calculada como a soma de todos os valores dividida pelo número de observações, a média é sensível a valores extremos (*outliers*) e representa o centro de massa da distribuição (Triola, 2018). Para uma amostra de tamanho n , a

⁴ Benchmarks são indicadores utilizados para comparar um conjunto de dados com uma referência ou padrão observado.

média é dada por:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

- **Mediana (Md):** O valor que divide a distribuição ordenada ao meio, de modo que 50% das observações estão abaixo e 50% acima. A mediana é robusta a *outliers* e representa melhor o centro de distribuições assimétricas (Wilcox, 2012). Quando a média é substancialmente diferente da mediana, isso indica assimetria na distribuição.
- **Moda:** O valor mais frequente na distribuição. Em distribuições multimodais, podem existir múltiplos valores com frequências máximas locais (Triola, 2018).

2.11.2 Medidas de Dispersão

As medidas de dispersão quantificam a variabilidade ou espalhamento dos dados em torno da tendência central (Montgomery; Runger, 2018). As principais métricas incluem:

- **Desvio padrão (σ):** Mede a dispersão média dos valores em relação à média aritmética. Um desvio padrão elevado indica alta variabilidade nos dados (Triola, 2018). Para uma amostra, o desvio padrão amostral é:

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2}$$

- **Quartis (Q_1, Q_3):** Valores que dividem a distribuição ordenada em quatro partes iguais. O primeiro quartil (Q_1) corresponde ao percentil 25, e o terceiro quartil (Q_3) ao percentil 75 (Wilcox, 2012).
- **Amplitude interquartil, do Inglês *Interquartile Range (IQR)*:** Calculada como $IQR = Q_3 - Q_1$, representa a dispersão dos 50% centrais da distribuição, sendo robusta a valores extremos (Rousseeuw; Leroy, 1987). O IQR é amplamente utilizado para detecção de *outliers*.

2.11.3 Medidas de Forma da Distribuição

As medidas de forma caracterizam a geometria da distribuição de probabilidade dos dados (Joanes; Gill, 1998):

- **Assimetria (*skewness*):** Mede o grau e a direção da assimetria da distribuição em relação à média. Valores positivos indicam assimetria à direita (cauda longa no lado positivo), enquanto valores negativos indicam assimetria à esquerda (Joanes; Gill, 1998).

Para uma distribuição normal, a assimetria é zero. O coeficiente de assimetria de Fisher é dado por:

$$\gamma_1 = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \mu}{\sigma} \right)^3$$

- **Curtose (*kurtosis*):** Mede o grau de achatamento ou propensão a extremos da distribuição em relação à distribuição normal (Westfall, 2014). Valores elevados de curtose indicam a presença de caudas pesadas e maior concentração de valores extremos. A curtose excessiva é definida em relação à distribuição normal (curtose = 3):

$$\gamma_2 = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \mu}{\sigma} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}$$

2.11.4 Detecção de *Outliers*

Outliers são observações que se desviam significativamente do padrão geral dos dados, podendo resultar de erros de medição, variabilidade natural ou eventos raros (Hawkins, 1980). A identificação de *outliers* é crucial para garantir a qualidade dos dados e evitar distorções em análises estatísticas e modelos preditivos.

O método baseado na amplitude interquartil (IQR) é uma técnica robusta e amplamente utilizada (Tukey, 1977). Define-se como *outliers* os valores que satisfazem:

$$x < Q_1 - 1,5 \times \text{IQR} \quad \text{ou} \quad x > Q_3 + 1,5 \times \text{IQR}$$

Este método é não-paramétrico, não assumindo distribuição específica dos dados, e resistente a distorções causadas pelos próprios *outliers* (Rousseeuw; Leroy, 1987).

2.11.5 Testes de Normalidade

Muitos métodos estatísticos e modelos de aprendizado de máquina assumem que os dados seguem uma distribuição normal (gaussiana). Testar essa suposição é essencial para escolher técnicas apropriadas de análise e interpretação dos resultados (Razali; Wah, 2011).

O teste de normalidade de D'Agostino-Pearson (D'Agostino; Pearson, 1973) combina os testes de assimetria e curtose para avaliar a hipótese nula de que os dados provêm de uma distribuição normal. A estatística de teste é dada por:

$$K^2 = Z_{\gamma_1}^2 + Z_{\gamma_2}^2$$

onde Z_{γ_1} e Z_{γ_2} são as estatísticas padronizadas de assimetria e curtose, respectivamente. Sob a hipótese nula, K^2 segue uma distribuição qui-quadrado com 2 graus de liberdade. Um valor-p inferior a um nível de significância predefinido (comumente $\alpha = 0,05$) indica rejeição da hipótese de normalidade.

Este teste é particularmente adequado para amostras de tamanho médio a grande ($n > 20$) e possui boa capacidade para detectar desvios da normalidade (Razali; Wah, 2011).

2.11.6 Teste de Heterogeneidade entre Grupos: ANOVA

A **análise de variância (ANOVA, do inglês *Analysis of Variance*)** é uma técnica estatística utilizada para comparar as médias de três ou mais grupos independentes, testando a hipótese nula de que todas as médias populacionais são iguais (Fisher, 1925). O teste ANOVA de uma via (*one-way ANOVA*) decompõe a variância total dos dados em duas componentes: variância entre grupos e variância dentro dos grupos.

A estatística F é calculada como:

$$F = \frac{\text{Variância entre grupos}}{\text{Variância dentro dos grupos}} = \frac{MS_{\text{entre}}}{MS_{\text{dentro}}}$$

onde MS_{entre} é a média quadrática entre grupos e MS_{dentro} é a média quadrática dentro dos grupos. Sob a hipótese nula de médias iguais, a estatística F segue uma distribuição F de *Fisher-Snedecor* com graus de liberdade apropriados. Um *valor-p* baixo ($p < 0,05$) indica evidência estatística de que pelo menos um grupo possui média significativamente diferente dos demais (Montgomery; Runger, 2018).

O teste ANOVA é especialmente relevante em contextos de aprendizado de máquina quando se deseja avaliar a heterogeneidade⁵ de características entre diferentes subgrupos (ex. projetos de software), justificando estratégias de validação cruzada específicas, como *Leave-One-Group-Out* (Raschka, 2018).

2.11.7 Análise de Correlação

A correlação mede a força e a direção da relação linear entre duas variáveis quantitativas (Cohen, 2009). O **coeficiente de correlação de Pearson** (ρ) é a métrica mais utilizada,

⁵ Neste trabalho, o termo *heterogeneidade* é utilizado para descrever a presença de variação sistemática entre os elementos do conjunto de dados, decorrente de diferenças entre projetos, domínios de aplicação, estilos de escrita e práticas de estimativa de esforço, e não apenas de flutuações aleatórias nos valores de *story points*.

variando entre -1 e $+1$:

$$\rho = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

onde x_i e y_i são os valores das variáveis, e $\bar{x}$ e $\bar{y}$ são suas respectivas médias.

Valores de $|\rho|$ próximos de 1 indicam forte correlação linear, enquanto valores próximos de 0 indicam ausência de relação linear. A interpretação típica segue (Mukaka, 2012):

- $|\rho| < 0,3$: correlação fraca
- $0,3 \leq |\rho| < 0,7$: correlação moderada
- $|\rho| \geq 0,7$: correlação forte

É importante ressaltar que correlação não implica causalidade, e o coeficiente de Pearson assume relação linear entre as variáveis (Wilcox, 2012). Para relações não-lineares ou dados ordinais, alternativas como o coeficiente de Spearman são mais apropriadas.

2.11.8 Implicações para Modelagem de Aprendizado de Máquina

A análise exploratória detalhada dos dados, empregando as métricas e testes estatísticos descritos, possui implicações diretas para o desenvolvimento e avaliação de modelos de aprendizado de máquina (Kuhn; Johnson, 2013):

- **Distribuições não-normais:** Justificam o uso de métricas de erro robustas a *outliers*, como o erro absoluto médio (MAE) e o erro absoluto mediano (MdAE), em detrimento do erro quadrático médio (MSE), que é altamente sensível a valores extremos (Willmott; Matsuura, 2005).
- **Heterogeneidade entre grupos:** Detectada por testes ANOVA, justifica estratégias de validação cruzada que respeitam a estrutura de grupos, como *Leave-One-Group-Out* ou *stratified k-fold*, evitando viés de generalização (Raschka, 2018).
- **Assimetria e curtose elevadas:** Indicam necessidade de transformações de dados (ex. logarítmica, Box-Cox) ou uso de algoritmos robustos a distribuições não-gaussianas (Osborne, 2002).
- **Correlações fracas:** Entre variáveis preditoras e variável-alvo sugerem que características superficiais são insuficientes, o que motiva o uso de representações mais sofisticadas, como *embeddings* profundos ou *feature engineering* avançado (Bengio; Courville; Vincent, 2013b).

Portanto, a aplicação de técnicas de análise exploratória, fundamentadas em princípios estatísticos sólidos, constitui pré-requisito essencial para o desenvolvimento de modelos preditivos robustos, interpretáveis e generalizáveis.

2.11.9 Métricas de Avaliação para Modelos de Regressão

Em problemas de regressão, onde o objetivo é prever valores contínuos, diversas métricas são utilizadas para avaliar o desempenho dos modelos preditivos. A escolha adequada dessas métricas é importante, pois diferentes métricas possuem sensibilidades distintas a características dos dados, como presença de *outliers*, assimetria e escala dos valores (Willmott; Matsuura, 2005).

2.11.9.1 Erro Absoluto Médio

O **Erro Absoluto Médio, do Inglês *Mean Absolute Error (MAE)*** mede a magnitude média dos erros de previsão sem considerar sua direção (Willmott; Matsuura, 2005). É calculado como a média das diferenças absolutas entre os valores previstos e os valores reais:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

onde y_i representa o valor real, $\hat{y}_i$ o valor previsto, e n o número de observações.

O MAE possui interpretação intuitiva, pois está na mesma unidade da variável-alvo, e é robusto a *outliers*, tratando todos os erros de forma linear (Chai; Draxler, 2014). Segundo Willmott e Matsuura (2005), o MAE é preferível ao MSE quando se deseja uma métrica que não seja excessivamente influenciada por valores extremos, sendo particularmente adequado para distribuições assimétricas ou com caudas pesadas.

2.11.9.2 Erro Quadrático Médio

O **Erro Quadrático Médio, do Inglês *Mean Squared Error (MSE)***, calcula a média dos quadrados das diferenças entre valores previstos e reais (James *et al.*, 2013):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Ao elevar os erros ao quadrado, o MSE penaliza desproporcionalmente erros maiores, tornando-se altamente sensível a *outliers* (Chai; Draxler, 2014). Esta característica pode ser vantajosa quando grandes erros são particularmente indesejáveis, mas problemática em dados com valores extremos frequentes ou assimetria acentuada (Willmott; Matsuura, 2005).

O MSE é amplamente utilizado em otimização de modelos devido às suas propriedades matemáticas convenientes (diferenciabilidade, convexidade em modelos lineares), sendo a função de perda padrão em muitos algoritmos de aprendizado de máquina (Hastie; Tibshirani; Friedman, 2009). Entretanto, sua unidade é o quadrado da variável-alvo, dificultando a interpretação direta.

2.11.9.3 Erro Absoluto Mediano

O **Erro Absoluto Mediano**, do Inglês *Median Absolute Error (MdAE)*, é a mediana das diferenças absolutas entre valores previstos e reais (Huber; Ronchetti, 2009):

$$\text{MdAE} = \text{mediana}(|y_1 - \hat{y}_1|, |y_2 - \hat{y}_2|, \dots, |y_n - \hat{y}_n|)$$

Como métrica baseada na mediana, o MdAE é robusto a *outliers*, sendo praticamente insensível a valores extremos (Rousseeuw; Leroy, 1987). Esta propriedade torna o MdAE especialmente adequado para avaliação de modelos em dados com presença significativa de anomalias, distribuições altamente assimétricas ou valores atípicos frequentes.

3 TRABALHOS RELACIONADOS

3.1 Modelos de Estimativa de Esforço de Software por Analogia (EESA)

A EESA é uma tarefa crítica no gerenciamento de projetos de software. Modelos preditivos têm sido amplamente explorados para melhorar a precisão dessas estimativas. Pode-se dizer que a EESA é ainda mais difícil de ser obtido na fase inicial de desenvolvimento, pois nessa fase, tudo o que se tem são informações textuais sobre os requisitos, normalmente no formato de histórias de usuário ou outro similar. Desta forma, usar esses textos como insumo para gerar estimativas confiáveis por analogia, é uma tarefa complexa.

Poucos trabalhos exploraram textos a fim de gerar estimativas de esforço de software, mas é importante destacar alguns deles, conforme apresentados no Quadro 1.

O trabalho de Choetkiertikul et al. (Choetkiertikul *et al.*, 2019) propôs o modelo Deep-SE para estimar esforço de software por analogia a partir de histórias de usuário. A abordagem combina LSTM e Recurrent Highway Networks (RHN), utilizando apenas uma etapa simples de pré-processamento (concatenação de título e descrição) e *word embeddings* sem contexto como entrada. Em cenário de estimativa entre projetos, o modelo obteve MAE de $3,82 \pm 1,56$, mas estudos subsequentes apontam como limitação central o uso de embeddings sem contexto, que dificulta capturar informações contextuais e lidar com polissemia em requisitos textuais curtos.

Um dos modelos recentes mais relevantes voltados à EESA é o SE³M (Fávero; Casanova; Pimentel, 2022). Trata-se de um modelo que utiliza representações textuais geradas por modelos pré-treinados, combinando técnicas de aprendizado profundo com PLN para estimar esforço de software a partir de requisitos. O SE³M se destaca por sua capacidade de integrar dados textuais de diferentes projetos de código aberto e ajustar-se a diferentes contextos de desenvolvimento de software. Nesse trabalho, foi utilizado o modelo pré-treinado BERT (Devlin *et al.*, 2018) para a representação textual dos requisitos de software contidos na base de dados proposta por Choetkiertikul *et al.* (2019), cujos *embeddings* foram submetidos a uma arquitetura de rede neural com saída linear. Os resultados experimentais relatados em Fávero, Casanova e Pimentel (2022) indicam que o SE³M apresenta desempenho superior a abordagens tradicionais de EESA baseadas em atributos manuais e modelos de aprendizado de máquina clássicos.

Dando continuidade a essa linha de pesquisa, o trabalho de Baratto (2022) comparou explicitamente modelos pré-treinados GPT-3 e BERT na Estimativa de Esforço de Software por Analogia a partir de requisitos textuais. O autor utilizou um modelo contextualizado GPT-3, que gera *embeddings* de alta dimensão para cada sentença e permite empregar apenas camadas densas (FNN), dispensando arquiteturas recorrentes. No experimento multi-repositórios (E1), o modelo denominado como Z-SE² obteve o melhor desempenho geral (MAE de $3,67 \pm 0,12$ e MdAE de 1,88), superando o SE³M baseado em BERT, e, no cenário *cross-repository* (E2), apresentou menores valores de MAE em 15 dos 16 subconjuntos de teste. Por outro lado, a

principal limitação apontada para o uso do GPT-3 é o custo elevado associado tanto à extração das representações textuais quanto a eventuais etapas de ajuste-fino, o que mantém o SE³M como uma alternativa competitiva e mais acessível em cenários com restrições de recursos.

3.2 Classificação de Texto com GCNs

A classificação de texto é uma das tarefas mais importantes em PLN. Recentemente, as GCNs têm se mostrado promissoras para essa tarefa devido à sua capacidade de capturar estruturas relacionais ricas e preservar a informação estrutural global do conjunto de dados.

Han *et al.* (2022) conduzem uma análise abrangente das técnicas de construção de grafos e dos mecanismos de aprendizado de GCNs na classificação de texto. Os autores destacam a importância das representações de nós e arestas na construção do grafo e como diferentes abordagens de aprendizado de GCN podem impactar o desempenho na classificação de texto. A pesquisa deles revela que a qualidade do grafo de entrada é crucial para o sucesso dos métodos baseados em GCN.

O trabalho de Yao, Mao e Luo (2018) introduz o TextGCN, um modelo de classificação de texto baseado em grafos que constrói um grafo textual a partir de um corpus inteiro. Neste modelo, palavras e documentos são representados como nós, e as relações de coocorrência entre palavras e documentos são representadas como arestas. O TextGCN utiliza GCNs para aprender as representações dos nós e realizar a classificação. Este modelo se destaca por sua capacidade de preservar as relações semânticas globais no grafo.

O BertGCN, proposto por Lin *et al.* (2022), combina o poder de pré-treinamento em larga escala do BERT com o aprendizado transdutivo das GCNs para a classificação de texto. O BertGCN constrói um grafo heterogêneo sobre o conjunto de dados, representando documentos como nós utilizando representações BERT. Ao treinar conjuntamente os módulos BERT e GCN, o BertGCN aproveita as vantagens de ambos os mundos, alcançando performances de estado da arte em uma ampla gama de conjuntos de dados de classificação de texto.

Liu *et al.* (2020) introduziram o *Tensor Graph Convolutional Networks* (TensorGCN), uma abordagem que integra informações heterogêneas de diferentes tipos de grafos. O TensorGCN constrói um tensor de grafos textuais que descrevem informações semânticas, sintáticas e contextuais sequenciais. Esse modelo realiza duas propagações: intra-grafo, para agregar informações de vizinhos dentro de um único grafo, e inter-grafo, para harmonizar informações entre diferentes grafos. Resultados experimentais em conjuntos de dados do tipo *benchmark* mostraram a eficácia do TensorGCN na classificação de textos, superando métodos tradicionais.

Os trabalhos relacionados revisados neste capítulo destacam o potencial das GCNs para melhorar a classificação de texto e, com isso, a estimativa de esforço de software. A combinação de técnicas de aprendizado profundo com representações baseadas em grafos tem mostrado resultados promissores, indicando direções futuras para pesquisas nesta área.

3.3 Posicionamento do Trabalho Proposto

Este trabalho diferencia-se dos trabalhos relacionados apresentados ao combinar elementos de ambas as linhas de pesquisa (estimativa de esforço de software por analogia (EESA) e classificação de texto com GCNs) propondo uma abordagem híbrida ainda não explorada na literatura de engenharia de software. Especificamente, este estudo adapta a arquitetura BertGCN (Lin *et al.*, 2022), originalmente desenvolvida para classificação de texto, ao contexto de regressão de *story points*, integrando representações contextualizadas via BERT com propagação estrutural em grafos heterogêneos.

Enquanto os trabalhos de EESA (Deep-SE, SE³M, Z-SE²) focam exclusivamente em arquiteturas baseadas em redes neurais sequenciais (LSTM, RHN) ou densas (FNN) aplicadas diretamente sobre *embeddings* de documentos, o modelo proposto introduz uma dimensão estrutural ao incorporar explicitamente relações de coocorrência documento-palavra (via TF-IDF) e palavra-palavra (via PPMI) em um grafo heterogêneo. Esta representação permite que a rede neural propague informações contextuais não apenas dentro de cada requisito individualmente, mas entre requisitos semanticamente relacionados através de palavras compartilhadas, potencialmente capturando padrões de complexidade e esforço que transcendem o conteúdo textual isolado de cada documento.

Em relação aos trabalhos de GCNs para classificação de texto (TextGCN, BertGCN, TensorGCN), a principal contribuição deste trabalho reside na adaptação da arquitetura para tarefa de regressão em um domínio altamente especializado (requisitos de software com *story points* associados) onde características como heterogeneidade entre projetos, distribuição assimétrica de valores de esforço e presença de jargões técnicos impõem desafios distintos daqueles encontrados em conjuntos de dados de classificação de texto genéricos. Adicionalmente, este trabalho investiga sistematicamente o impacto de duas estratégias de pré-processamento textual: (1) tradicional, baseada em limpeza e normalização lexical; e (2) refinamento via modelo de linguagem de grande escala (GPT-5 mini), que reestrutura os requisitos no formato canônico de *user stories*, visando avaliar se a padronização narrativa aprimora a qualidade das representações semânticas e, conseqüentemente, a acurácia das estimativas de esforço.

Esta convergência entre técnicas de aprendizado profundo em grafos e estimativa de esforço de software por analogia configura uma contribuição relevante, ao ampliar o repertório de abordagens para o desafio da estimativa confiável de esforço em projetos de desenvolvimento de software.

Quadro 1 – Trabalhos relacionados à estimativa de esforço

ID da publicação	Tipo de análise textual (dados textuais utilizados)	Técnica(s) de representação textual	Modelo(s) de representação textual	Pré-processamento
(Choetkiertikul <i>et al.</i> , 2019)	Semântica	<i>Word embedding</i> sem contexto a nível de palavra + representação vetorial a nível de texto (via LSTM)	<i>Word embedding</i>	Apenas uma fase de pré-processamento de dados brutos; combinação de título e descrição em uma única sentença.
(Fávero; Casanova; Pimentel, 2022)	Semântica	<i>Word embedding</i> com e sem contexto a nível de palavra + representação vetorial a nível de texto (via LSTM)	<i>Word embedding</i>	Conversão para minúsculas; remoção de caracteres especiais e números; remoção de <i>stopwords</i> ; tokenização.
(Baratto, 2022)	Semântica	<i>Word embedding</i> via GPT-3	<i>Word embedding</i>	Conversão para minúsculas; remoção de caracteres especiais e números; remoção usando <i>stoplist</i> (incluindo tags HTML); tokenização; uso de expressões regulares.
Este trabalho	Semântica	<i>Word embedding</i> contextualizado (via BERT) + representação textual por grafos (via GCN, TF-IDF e PPMI)	BertGCN	Duas estratégias: (1) tradicional (concatenação, RegEx, minúsculas, tokenização via <i>BertTokenizer</i>); (2) refinamento via LLM (GPT-5 mini).

Fonte: Adaptado de (Fávero; Casanova; Pimentel, 2022).

4 MATERIAIS E MÉTODOS

Este capítulo descreve os materiais e métodos que serão utilizados para conduzir esta pesquisa. Serão apresentados o conjunto de dados, o modelo de aprendizado de máquina que será utilizado, o processo de treinamento, as métricas de avaliação e as ferramentas e ambiente de desenvolvimento.

4.1 Materiais

No quadro 2, são apresentadas as tecnologias necessárias para o desenvolvimento do trabalho.

Quadro 2 – Materiais Utilizados no Desenvolvimento do trabalho

Ferramenta	Versão	Disponível em	Finalidade
Python	3.14.0	https://www.python.org/	Linguagem de programação utilizada para desenvolver os modelos
Google Colab	N/A	https://colab.research.google.com/	Plataforma para desenvolvimento e execução de código em ambiente de nuvem
Transformers	4.57.1	https://huggingface.co/transformers/	Biblioteca utilizada para acessar modelos pré-treinados BERT e outros modelos baseados em <i>transformers</i>
PyTorch	2.9.1	https://pytorch.org/	Biblioteca de <i>deep learning</i> utilizada para construção e treinamento dos modelos
Torch-Geometric	2.7.0	https://pytorch-geometric.readthedocs.io/	Biblioteca para aprendizado em grafos (<i>Graph Neural Networks</i>)
Scikit-learn	1.7.2	https://scikit-learn.org/	Biblioteca para manipulação de dados e cálculo de métricas
Pandas	2.3.3	https://pandas.pydata.org/	Biblioteca para manipulação e análise de dados
OpenAI Python	2.8.0	https://pypi.org/project/openai/	Biblioteca utilizada para acesso à API do <i>GPT-5 mini</i> , empregada no refinamento textual dos requisitos
BERT	base-uncased	https://huggingface.co/bert-base-uncased	Modelo pré-treinado utilizado para obtenção de <i>embeddings</i> textuais

Fonte: Autoria própria (2025).

4.1.1 Base de Dados

A base de dados utilizada nesta pesquisa consiste de requisitos textuais de software, no formato de histórias de usuários, cada um com seu respectivo esforço de desenvolvimento em *story points*. Essa mesma base foi utilizada por (Choetkiertikul *et al.*, 2019) e, posteriormente, por (Fávero; Casanova; Pimentel, 2022).

Os dados foram coletados de diversos projetos de código aberto, em um total de 16 projetos, como mostra a tabela 1.

Tabela 1 – Descrição e número de requisitos textuais por projeto.

Número do projeto	Descrição	Requisitos/projeto
0	Mesos	1680
1	Usergrid	482
2	Appcelerator Studio	2919
3	Aptanastudio	829
4	Titanium	2251
5	DuraCloud	666
6	Bamboo	521
7	Clover	384
8	JIRA Software	352
9	Moodle	1166
10	Data Management	4667
11	Mule	889
12	Mule Studio	732
13	Spring XD	3526
14	Talend Data Quality	1381
15	Talend ESB	868
Total		23.313

Fonte: (Choetkiertikul *et al.*, 2019).

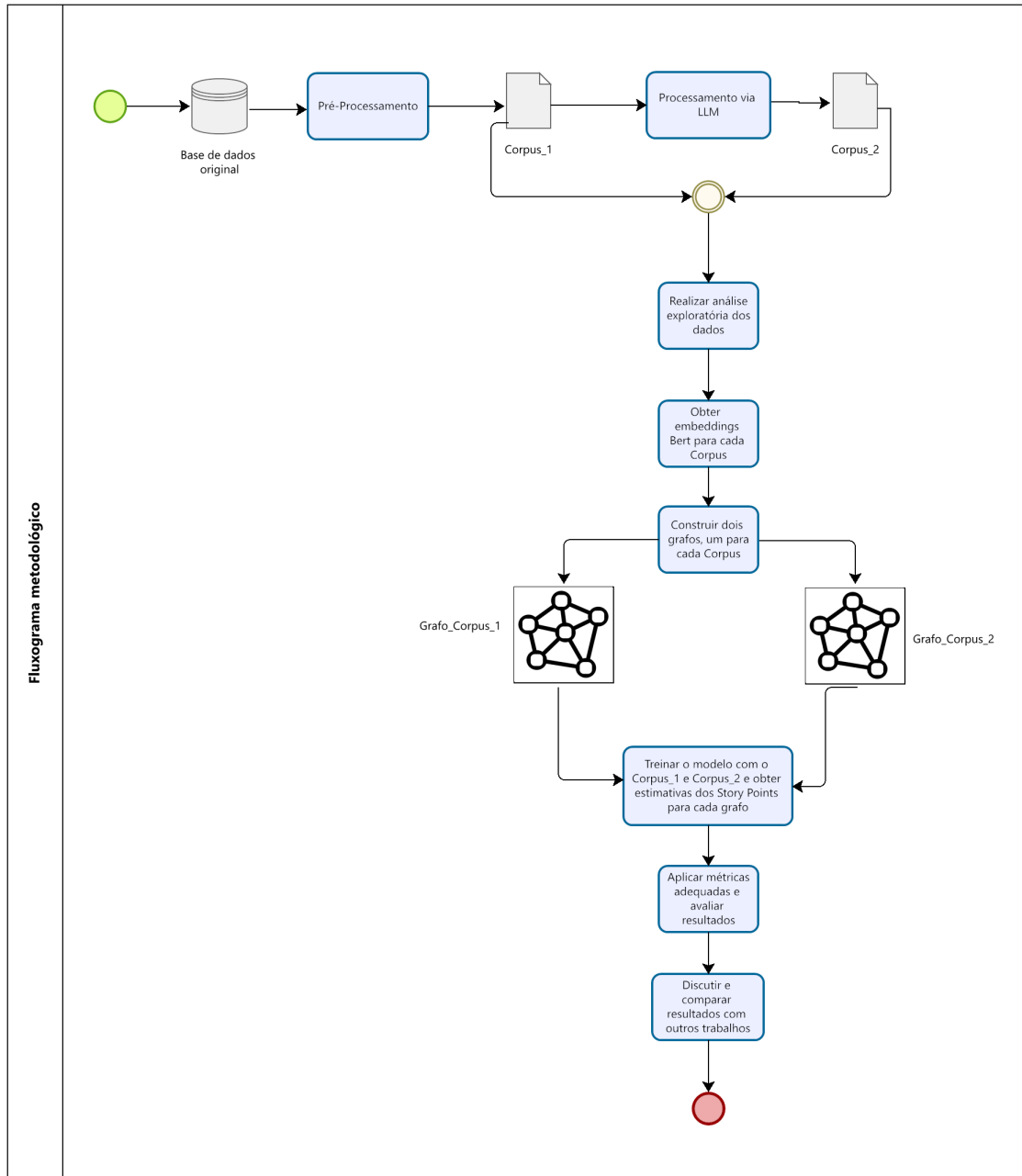
4.2 Método

A seguir, são apresentadas as etapas que compõem o método utilizado nesta pesquisa, detalhando cada fase do processo desde o pré-processamento da base de dados original até a avaliação final do modelo. O fluxograma da Figura 7 resume essas etapas, destacando a geração dos dois *corpora*, a construção dos grafos heterogêneos e o treinamento do modelo BertGCN.

4.2.1 Análise Exploratória dos Dados

Antes de proceder com a aplicação dos modelos de aprendizado de máquina, realiza-se uma análise exploratória detalhada do conjunto de dados para compreender suas características fundamentais, identificar padrões, *outliers* e peculiaridades que possam influenciar o

Figura 7 – Fluxograma metodológico da pesquisa, contemplando a geração dos *corpora*, a construção dos grafos heterogêneos e o treinamento e avaliação do modelo BertGCN.



Fonte: Autoria própria (2025).

desempenho preditivo dos modelos. Esta etapa é crucial para validar a qualidade dos dados, justificar decisões metodológicas e contextualizar os resultados obtidos em relação à literatura existente (Fávero; Casanova; Pimentel, 2022).

O conjunto de dados compreende 23.313 requisitos de software (*user stories*) distribuídos entre 16 projetos de código aberto. A análise exploratória investiga quatro dimensões principais:

- **Estatísticas descritivas gerais:** Medidas de tendência central (média, mediana) e dispersão (desvio padrão, amplitude interquartil) dos *story points*, identificação de valores extremos e caracterização da distribuição (assimetria, curtose).
- **Distribuição de *story points*:** Análise da forma da distribuição (assimetria positiva/negativa), concentração de valores em classes específicas, detecção de *outliers* pelo método IQR (*Interquartile Range*) e sua influência potencial na modelagem.
- **Heterogeneidade entre projetos:** Comparação de estatísticas descritivas segregadas por projeto, teste ANOVA para verificar significância estatística das diferenças interprojetos, identificação de projetos com comportamento atípico que justifiquem metodologias de validação *cross-repository*.
- **Características textuais dos requisitos:** Análise do comprimento textual (em palavras), correlações entre comprimento textual e *story points*.

Os resultados da análise exploratória são apresentados no Capítulo 5, Seção 5.1, incluindo tabelas de estatísticas descritivas, gráficos de distribuição e testes estatísticos que fundamentam as escolhas metodológicas subsequentes.

4.2.2 Pré-processamento dos Dados

O pré-processamento dos dados textuais é uma etapa fundamental para garantir a qualidade das representações semânticas e, conseqüentemente, o desempenho dos modelos preditivos. Neste trabalho, foram aplicadas duas estratégias de pré-processamento, gerando dois *corpora* distintos para análise comparativa.

4.2.2.1 Corpus_1: Pré-processamento Tradicional

O *Corpus_1* foi gerado por meio da aplicação de técnicas convencionais de limpeza e normalização textual, amplamente utilizadas na literatura de processamento de linguagem natural (Manning; Schütze, 1999). O processamento seguiu as seguintes etapas:

1. **Concatenação de campos:** Os campos *title* e *description* foram concatenados para formar um único texto representativo de cada requisito, garantindo que todas as informações relevantes fossem preservadas.

2. **Remoção de caracteres especiais:** Caracteres não alfanuméricos (exceto espaços) foram removidos utilizando expressões regulares (Regex), eliminando ruídos como pontuações, símbolos e caracteres de controle.
3. **Normalização de espaços:** Espaços múltiplos foram substituídos por espaços únicos, e espaços iniciais ou finais foram removidos para garantir consistência.
4. **Conversão para minúsculas:** Todos os caracteres foram convertidos para letras minúsculas para evitar que variações de capitalização fossem interpretadas como palavras distintas.
5. **Tokenização:** A tokenização foi aplicada a nível de requisito utilizando o *BertTokenizer*, dividindo os textos em unidades menores (tokens) que podem ser processadas pelo modelo BERT pré-treinado.

O *Corpus_1* preserva a estrutura linguística original dos requisitos, aplicando apenas transformações básicas de normalização. Esta abordagem é consistente com a maioria dos trabalhos da literatura em estimativa de esforço por analogia (Choetkiertikul *et al.*, 2019; Fávero; Casanova; Pimentel, 2022), permitindo comparação direta com os resultados reportados.

4.2.2.2 Corpus_2: Refinamento via Modelo de Linguagem

O *Corpus_2* foi gerado através do refinamento do *Corpus_1* utilizando o modelo de linguagem GPT-5 mini, acessado via OpenAI API (OpenAI, 2025), com o objetivo de aprimorar a qualidade semântica e estrutural dos requisitos. A estratégia de refinamento consistiu em solicitar ao modelo que:

1. Corrigisse inconsistências gramaticais e ortográficas presentes nos requisitos originais;
2. Removesse ruídos textuais, como caracteres especiais e trechos irrelevantes, tornando os requisitos mais claros e concisos;
3. Reformulasse descrições ambíguas ou fragmentadas, mantendo fidelidade ao significado técnico original;
4. Padronizasse, sempre que possível, a estrutura narrativa dos requisitos seguindo o formato canônico¹ de *user stories*.

O refinamento via *Large Language Model* (LLM) visa mitigar limitações comuns em requisitos de software, como descrições fragmentadas e inconsistências linguísticas, que podem

¹ No desenvolvimento ágil, *user stories* são frequentemente escritas em um formato canônico do tipo: “Como [tipo de usuário], eu quero [funcionalidade] para [benefício/resultado]”, o que facilita a clareza de papel, objetivo e valor de negócio da demanda (Cohn, 2004; Rubin, 2012).

prejudicar a qualidade das representações semânticas. A comparação entre os desempenhos obtidos com `Corpus_1` e `Corpus_2` permite avaliar o impacto do processamento avançado via modelos de linguagem na tarefa de estimativa de esforço.

Exemplos comparativos entre requisitos processados pelos dois métodos são apresentados no Capítulo 5, Seção 5.2, ilustrando as diferenças qualitativas introduzidas pelo refinamento via LLM.

4.2.3 Representação via BertGCN

Nesta pesquisa, é implementada a arquitetura do modelo BertGCN proposta por (Lin *et al.*, 2022), aplicando o conceito de aprendizado transdutivo para gerar estimativa de esforço de software.

Para a representação da estrutura textual da base de dados via BertGCN é necessário seguir as seguintes etapas:

- **Construção do grafo:** Um grafo heterogêneo é construído contendo nós que representam tanto documentos quanto palavras, seguindo a abordagem do TextGCN (Yao; Mao; Luo, 2018). No contexto deste trabalho, cada documento corresponde a um requisito textual da base de dados de Choetkiertikul *et al.* (2019). A Figura 8 ilustra, de forma esquemática, esse tipo de grafo: os nós em azul representam requisitos (documentos) e os nós em vermelho representam palavras, enquanto as arestas conectando requisitos e palavras correspondem às ocorrências de termos nos documentos.

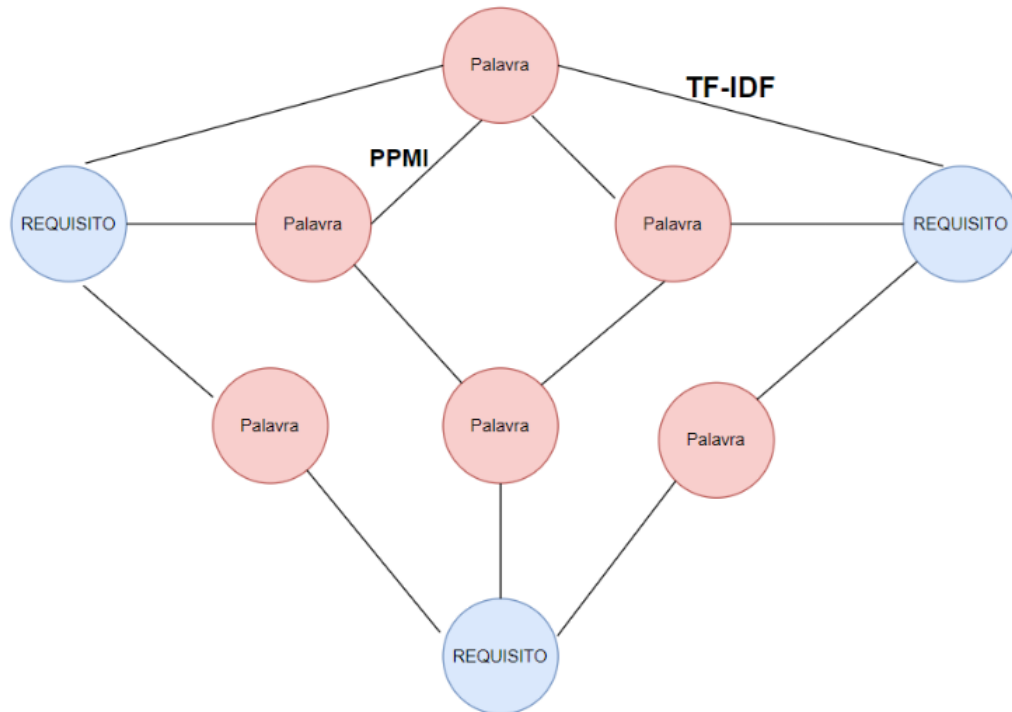
As arestas entre documentos e palavras são ponderadas pela frequência inversa do termo (TF-IDF), refletindo a importância relativa de cada palavra em cada requisito. Já as arestas entre palavras são ponderadas pela *Positive Pointwise Mutual Information* (PPMI), que captura a força de coocorrência entre pares de termos ao longo do corpus. Formalmente, o peso de uma aresta entre dois nós i e j é definido como:

$$A_{i,j} = \begin{cases} \text{PPMI}(i, j), & \text{se } i, j \text{ são palavras e } i \neq j \\ \text{TF-IDF}(i, j), & \text{se } i \text{ é documento e } j \text{ é palavra} \\ 1, & \text{se } i = j \\ 0, & \text{caso contrário.} \end{cases} \quad (1)$$

Essa estrutura permite capturar, em um único grafo, tanto a semântica distribuída das palavras quanto as relações estruturais entre requisitos e termos.

- **Inicialização dos *embeddings*:** As representações dos documentos são inicializadas com *embeddings* obtidos de um modelo BERT pré-treinado. A Figura 9 ilustra o fluxo

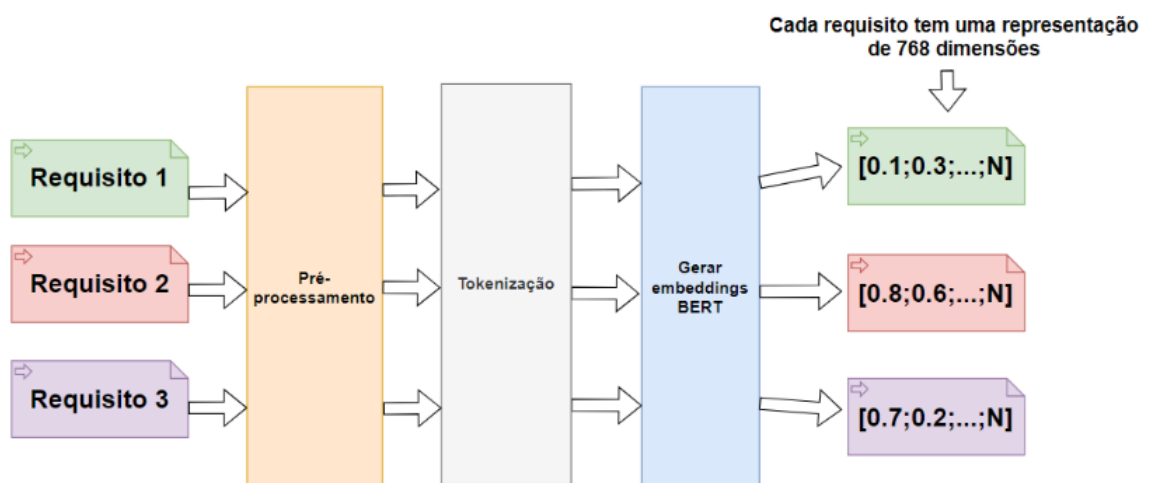
Figura 8 – Exemplo de grafo heterogêneo documento–palavra utilizado na construção do TextGCN/BertGCN, em que nós azuis representam requisitos (documentos), nós vermelhos representam palavras, as arestas documento–palavra são ponderadas por TF-IDF e as arestas palavra–palavra por PPMI.



Fonte: Autoria própria (2025).

de pré-processamento, tokenização e geração de *embeddings* BERT, no qual cada requisito é mapeado para um vetor denso de 768 dimensões.

Figura 9 – Fluxo de pré-processamento, tokenização e geração de *embeddings* BERT para cada requisito.



Fonte: Autoria própria (2025).

A matriz de características iniciais X é dada por:

$$X = \begin{pmatrix} X_{doc} \\ 0 \end{pmatrix}_{(n_{doc}+n_{word}) \times d} \quad (2)$$

onde X_{doc} são os *embeddings* dos documentos e d é a dimensionalidade dos *embeddings*. A Figura 10 ilustra um exemplo de uma matriz de características representando dois documentos, e 3 palavras, a representação dos documentos são inicializados com os *embeddings* que no caso do exemplo é de dimensão 4.

Figura 10 – Exemplo de uma matriz de características inicial.

$$X = \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{matrix} \text{Documento 1} \\ \text{Documento 2} \\ \text{Palavra 1} \\ \text{Palavra 2} \\ \text{Palavra 3} \end{matrix}$$

Fonte: Autoria própria (2025)

- Propagação no GCN: A matriz de características X é alimentada em um modelo GCN (Kipf; Welling, 2017) que propaga iterativamente mensagens entre os exemplos de treino e teste. Os *embeddings* são passados por camadas de GCN para atualizar as representações dos nós com base na estrutura do grafo. Especificamente, a matriz de características de saída da i -ésima camada GCN $L^{(i)}$ é computada como:

$$L^{(i)} = \rho(\tilde{A}L^{(i-1)}W^{(i)}) \quad (3)$$

onde ρ é uma função de ativação, $\tilde{A}$ é a matriz de adjacência normalizada e $W^{(i)}$ é a matriz de pesos da camada. $L^{(0)} = X$ é a matriz de características de entrada do modelo. A saída final do GCN, L_{final} , é então alimentada na camada de regressão para prever os valores de esforço (*story points*). A predição do esforço é dada por:

$$\text{Esforço Predito} = g(L_{final})$$

onde g representa a camada de regressão que transforma as representações finais dos documentos em valores preditos de esforço.

4.2.4 Particionamento dos dados

A partição do conjunto de dados nas fases de treinamento, validação e avaliação do modelo foi realizada por meio do método *k-fold* aninhado. O método *k-fold* aninhado (do inglês, *nested k-fold cross-validation*) é uma técnica de validação cruzada que envolve dois níveis de validação *k-fold* para ajustar hiperparâmetros e avaliar o desempenho do modelo de forma robusta (Cawley; Talbot, 2010). Esse procedimento ajuda a evitar *overfitting* nos hiperparâmetros, proporcionando uma estimativa mais precisa da performance do modelo.

O método *nested k-fold* consiste em dois *loops* de validação cruzada:

- **Loop externo (avaliação):** divide os dados em k_1 *folds* para avaliar o desempenho final do modelo de forma imparcial. Cada *fold* externo serve como conjunto de teste, enquanto os demais são utilizados para seleção de hiperparâmetros e treinamento.
- **Loop interno (seleção de hiperparâmetros):** para cada *fold* externo, realiza validação cruzada com k_2 *folds* nos dados de treino do *loop* externo para buscar os melhores hiperparâmetros por meio de *grid search*. Os hiperparâmetros que minimizam o erro médio no *loop* interno são então utilizados para treinar o modelo final.

Essa metodologia garante que os hiperparâmetros sejam otimizados sem “ver” os dados de teste do *loop* externo, evitando vazamento de informação e proporcionando estimativas imparciais de generalização (Fávero; Casanova; Pimentel, 2022).

4.2.4.1 Experimentos Conduzidos

Foram conduzidos dois experimentos principais, variando a estratégia de particionamento no *loop* externo:

- **Experimento E1 (Multi-repositórios):** Utilizando a divisão original do método *k-fold* aninhado, com número de subconjuntos do laço externo $k_1 = 10$ e número de subconjuntos do laço interno $k_2 = 10$. Neste experimento, denominado **multi-projetos**, os conjuntos de treino e teste contêm amostras de requisitos provenientes de mais de um projeto, avaliando a capacidade do modelo de generalizar entre diferentes domínios quando treinado com dados heterogêneos.
- **Experimento E2 (Cross-repository):** Utilizando o particionamento por projetos por meio da metodologia *Leave-One-Project-Out (LOPO)*, onde o número de subconjuntos do laço externo $k_1 = 16$, corresponde ao número de projetos existentes no *corpus*. O *loop* interno mantém $k_2 = 10$ *folds* para seleção de hiperparâmetros. Neste experimento, denominado **cross-repository**, cada iteração do *loop* externo utiliza um projeto

completo como conjunto de teste e os demais 15 projetos para treinamento e validação, avaliando a capacidade do modelo de generalizar para projetos novos não vistos durante o treinamento.

Esta estratégia de particionamento segue rigorosamente a metodologia proposta por Fávero, Casanova e Pimentel (2022), permitindo comparação direta com os resultados da literatura e avaliação robusta da capacidade de generalização do modelo em diferentes cenários.

4.2.4.2 Considerações sobre Aprendizado Transdutivo

A abordagem de aprendizado transdutivo empregada neste trabalho, seguindo a arquitetura BertGCN original (Lin *et al.*, 2022), mantém todos os documentos no grafo heterogêneo durante o treinamento, utilizando máscaras para separar conjuntos de treino, validação e teste. Especificamente:

- O grafo heterogêneo global é construído uma única vez contendo todos os 23.313 requisitos e o vocabulário completo extraído do *corpus*.
- Durante o treinamento em cada *fold*, máscaras binárias (*train_mask*, *val_mask*, *test_mask*) indicam quais documentos pertencem a cada conjunto, mas todos os nós permanecem no grafo.
- A função de perda é calculada apenas sobre os documentos com *train_mask = True*, enquanto os documentos de validação e teste contribuem para a propagação de *features* via GCN, mas não para o gradiente.

Esta estratégia difere da abordagem indutiva tradicional empregada na literatura de estimativa de esforço (Choetkiertikul *et al.*, 2019; Fávero; Casanova; Pimentel, 2022), onde o modelo é treinado exclusivamente em dados de treino e aplicado a dados completamente novos. A escolha pelo paradigma transdutivo permite avaliar a capacidade da GCN de propagar informação estrutural por meio do grafo heterogêneo, potencialmente capturando relações semânticas latentes entre requisitos de diferentes projetos.

Reconhece-se, entretanto, que esta abordagem pode resultar em estimativas potencialmente otimistas quando comparadas a cenários de produção, onde novos projetos não estão disponíveis durante o treinamento. Esta limitação metodológica é explicitamente documentada e discutida no Capítulo 5, incluindo propostas para versões indutivas do modelo como trabalho futuro.

4.2.5 Treinamento do Modelo

O treinamento do modelo BertGCN neste trabalho adapta a arquitetura original proposta por Lin *et al.* (2022) para a tarefa de regressão de *story points*, com foco exclusivo no caminho de propagação via GCN. A seguir, são descritas as etapas do processo de treinamento implementado, incluindo a justificativa para a escolha de $\lambda = 1.0$ e os detalhes da otimização das camadas de convolução em grafos.

4.2.5.1 Arquitetura do Modelo

A arquitetura BertGCN original proposta por Lin *et al.* (2022) combina dois caminhos complementares: um caminho GCN que propaga *features* através da estrutura do grafo e um caminho BERT que aplica classificação direta sobre os *embeddings*. A predição final é obtida pela combinação linear ponderada:

$$Z = \lambda \cdot Z_{GCN} + (1 - \lambda) \cdot Z_{BERT} \quad (4)$$

onde $\lambda \in [0, 1]$ controla a contribuição relativa de cada caminho.

4.2.5.1.1 Escolha de $\lambda = 1.0$: Foco exclusivo na GCN

Neste trabalho, optou-se por $\lambda = 1.0$, configurando o modelo para utilizar exclusivamente o caminho GCN (100% GCN, 0% BERT direto). Esta decisão metodológica visa avaliar especificamente a capacidade da propagação estrutural via grafos heterogêneos de capturar relações semânticas entre requisitos e palavras para a tarefa de estimativa de esforço.

As motivações para esta escolha incluem:

1. **Contribuição científica focada:** Isolar o efeito da propagação GCN permite avaliar objetivamente se a estrutura do grafo heterogêneo (arestas TF-IDF e PPMI) agrega valor preditivo além das representações semânticas BERT. Trabalhos anteriores (Fávero; Casanova; Pimentel, 2022; Choetkiertikul *et al.*, 2019) já demonstraram a eficácia de modelos baseados exclusivamente em *embeddings* textuais; este trabalho investiga se a topologia do grafo oferece vantagens adicionais.
2. **Captura de relações estruturais:** A propagação via GCN permite que documentos semanticamente relacionados mesmo sem co-ocorrência direta de palavras influenciem mutuamente suas representações, potencialmente capturando padrões de complexidade compartilhados entre requisitos similares de diferentes projetos.
3. **Consistência com a literatura de grafos:** A maioria dos trabalhos em classificação de textos com GCN (Yao; Mao; Luo, 2018; Liu *et al.*, 2020) utiliza exclusivamente a

propagação estrutural, sem componentes de ensemble ², demonstrando que a GCN por si só pode aprender representações eficazes quando o grafo é construído adequadamente.

Com $\lambda = 1.0$, a arquitetura implementada consiste exclusivamente de camadas de convolução em grafos (GCNConv) que propagam os *embeddings* BERT iniciais através da estrutura heterogênea documento-palavra. A arquitetura suporta número variável de camadas GCN ($N \in \{2, 3\}$) com a seguinte estrutura:

- **Inicialização:** Cada nó de documento é inicializado com seu *embedding* BERT ($dim = 768$), enquanto nós de palavras são inicializados com vetores zero.
- **Primeira camada GCN:** $768 \rightarrow hidden_dim$ (tipicamente 200 ou 256)
- **Camadas intermediárias:** $hidden_dim \rightarrow hidden_dim$ (se $num_layers > 2$)
- **Última camada GCN:** $hidden_dim \rightarrow 1$ (predição de *story points*)
- **Regularização:** Após cada camada intermediária, aplicam-se BatchNormalization, ativação ReLU e *dropout* para estabilizar o treinamento e prevenir *overfitting*.

A propagação em cada camada GCN segue a formulação de Kipf e Welling (2017):

$$L^{(i)} = \rho \left(\tilde{A} L^{(i-1)} W^{(i)} \right) \quad (5)$$

onde $L^{(i)}$ é a matriz de features da camada i , $\tilde{A}$ é a matriz de adjacência normalizada do grafo heterogêneo (contendo pesos TF-IDF e PPMI), $W^{(i)}$ são os pesos treináveis da camada e ρ é a função de ativação ReLU. A predição final para cada documento é obtida diretamente da última camada GCN: $\hat{y} = L_{doc}^{(N)}$.

4.2.5.2 Configuração de Hiperparâmetros

Os hiperparâmetros do modelo foram definidos através de busca em grade (*grid search*) no *loop* interno do *nested k-fold*, explorando quatro configurações principais baseadas na arquitetura original e variações sistemáticas:

1. **Configuração 1:** $hidden_dim = 200$, $dropout = 0.5$, $num_layers = 2$, $lr = 0.001$
2. **Configuração 2 (Alta capacidade):** $hidden_dim = 256$, $dropout = 0.3$, $num_layers = 2$, $lr = 0.001$

² *Ensemble* é uma estratégia que combina as predições de múltiplos modelos base para melhorar o desempenho e a robustez em relação a um único modelo (Breiman, 1996; Freund; Schapire, 1997; Wolpert, 1992). Exemplos comuns incluem *bagging* (Breiman, 1996), *boosting* (Freund; Schapire, 1997) e *stacking* (Wolpert, 1992).

3. **Configuração 3 (Convergência rápida):** $hidden_dim = 256$, $dropout = 0.3$, $num_layers = 2$, $lr = 0.005$

4. **Configuração 4 (Propagação profunda):** $hidden_dim = 256$, $dropout = 0.5$, $num_layers = 3$, $lr = 0.001$

Os demais parâmetros permaneceram fixos: dimensão de entrada $input_dim = 768$ (dimensão dos *embeddings* BERT-base-uncased), dimensão de saída $output_dim = 1$ (valor escalar de *story points*), $\lambda = 1.0$ e regularização $weight_decay = 10^{-3}$.

4.2.5.3 Processo de Treinamento

Dado que o modelo utiliza exclusivamente o caminho GCN ($\lambda = 1.0$), o treinamento concentra-se na otimização das camadas de convolução em grafos. O processo segue a estratégia proposta por Lin *et al.* (2022), adaptada para o cenário de regressão de *story points*:

4.2.5.3.1 Treinamento das Camadas GCN

Os *embeddings* BERT utilizados para inicializar os nós de documentos permanecem fixos durante todo o treinamento, servindo como features de entrada de alta qualidade para a propagação no grafo. Apenas os pesos das camadas GCN ($W^{(1)}, W^{(2)}, \dots, W^{(N)}$) e os parâmetros de BatchNormalization são otimizados. Esta abordagem possui as seguintes características:

- **Função de perda:** MSELoss (Mean Squared Error)
- **Otimizador:** Adam com taxa de aprendizado $lr = 0.001$ (ou valor do grid search) e regularização $weight_decay = 10^{-3}$
- **Número de épocas:** 300 épocas (*loop* externo - modelo final) ou 30 épocas (*loop* interno - busca de HP)
- **Gradiente clipping:** Norma máxima de 1.0 para prevenir explosão de gradientes
- **Early stopping:** Paciência de 50 épocas (*loop* externo) ou 15 épocas (*loop* interno), monitorando a perda de validação
- **Frequência de validação:** A cada 5 épocas (*loop* externo) ou 10 épocas (*loop* interno)

A cada iteração de treinamento, o modelo executa um *forward pass* propagando features através das camadas GCN, calcula a perda MSE apenas nos documentos de treino, e atualiza os pesos $W^{(l)}$ via *backpropagation*.

Durante o treinamento, apenas os gradientes correspondentes aos nós de documentos de treino (identificados por $train_mask = True$) são utilizados para calcular a função de perda:

$$\mathcal{L} = \frac{1}{|\mathcal{D}_{train}|} \sum_{i \in \mathcal{D}_{train}} (\hat{y}_i - y_i)^2 \quad (6)$$

onde $\mathcal{D}_{train}$ é o conjunto de índices de documentos de treino, $\hat{y}_i = L_i^{(N)}$ é a predição do modelo e y_i é o *story point* verdadeiro.

Importante ressaltar que, embora apenas os documentos de treino contribuam para o gradiente, todos os nós do grafo (incluindo documentos de validação, teste e nós de palavras) participam da propagação de features via GCN. Esta é a essência do aprendizado transdutivo empregado pela arquitetura BertGCN: o grafo global permanece intacto durante o treinamento, e máscaras binárias controlam quais documentos são utilizados para calcular a perda.

4.3 Inferência da Estimativa de Esforço de Software

Em resumo o modelo BertGCN utiliza uma combinação de *embeddings* BERT e GCN para prever os valores de esforço (*story points*) dos requisitos de software. Inicialmente, os textos dos requisitos são tokenizados e convertidos em *embeddings* utilizando o modelo BERT pré-treinado. Um grafo heterogêneo é então construído, onde os nós representam documentos e palavras, e as arestas são definidas com base no TF-IDF e PPMI.

Durante o treinamento, as características dos nós são propagadas por meio das camadas GCN. Cada camada aplica convoluções sobre os nós, utilizando a matriz de adjacência para propagar informações entre os nós conectados. As saídas das camadas GCN são representações atualizadas dos documentos, que são então usadas para prever os valores de esforço. A saída final do modelo fornece diretamente os valores estimados de esforço para cada documento, correspondendo aos *story points* previstos.

4.4 Métricas de Avaliação

Após o treinamento em cada *fold*, o modelo foi avaliado no conjunto de teste utilizando métricas como MAE, MSE, MdAE e desvio padrão.

4.4.1 Resultados e Análise

Os resultados obtidos pelo modelo serão apresentados e comparados com o modelo SE³M (Fávero; Casanova; Pimentel, 2022) e Z-SE² (Baratto, 2022), discutindo-se a eficácia e as possíveis melhorias entre as abordagens utilizadas.

5 RESULTADOS E DISCUSSÃO

Neste capítulo, são apresentados os resultados obtidos a partir da aplicação do modelo BertGCN na tarefa de estimativa de esforço de software utilizando requisitos textuais expressos em *user stories*. Os experimentos foram conduzidos em três etapas distintas: primeiro, uma análise exploratória detalhada do conjunto de dados para compreender suas características, distribuições e padrões intrínsecos; em seguida, a preparação e pré-processamento dos dados textuais; e, por fim, a aplicação e avaliação rigorosa do modelo proposto utilizando metodologias de validação cruzada *nested k-fold* e *LOPO*.

O conjunto de dados utilizado neste trabalho é derivado da base unificada empregada por Fávero, Casanova e Pimentel (2022), contendo 23.313 requisitos de software (*user stories*) oriundos de 16 projetos distintos de código aberto. Esta base foi submetida a dois níveis de pré-processamento: o *Corpus_1*, resultante de processamento textual tradicional com técnicas de limpeza, normalização e remoção de ruídos; e o *Corpus_2*, obtido pelo refinamento do *Corpus_1* por meio de um LLM (*GPT-5 mini*), visando aprimorar a qualidade semântica e estrutural dos requisitos. A comparação entre esses corpora permite avaliar não apenas o desempenho da arquitetura BertGCN, mas também o impacto qualitativo do processamento avançado via LLM na representação semântica dos requisitos e, conseqüentemente, na acurácia das estimativas de esforço.

A análise exploratória, apresentada na Seção 5.1, caracteriza estatisticamente o conjunto de dados, investigando a distribuição de *story points*, a heterogeneidade entre projetos, as características textuais dos requisitos e a presença de padrões ou anomalias que possam influenciar o desempenho dos modelos preditivos. Esta etapa é fundamental para justificar escolhas metodológicas, identificar desafios inerentes aos dados e contextualizar os resultados obtidos frente à literatura consolidada em estimativa de esforço de software.

Posteriormente, na Seção 5.4, são reportados os resultados quantitativos dos experimentos utilizando as métricas MAE, MSE e MdAE. A validação cruzada *nested k-fold* e a metodologia *LOPO* garantem robustez estatística às avaliações, permitindo comparação com os trabalhos de referência da literatura. A análise comparativa entre diferentes configurações de hiperparâmetros, arquiteturas de propagação e estratégias de construção do grafo heterogêneo permite identificar os fatores que impactam positivamente ou negativamente o desempenho do modelo BertGCN na tarefa de estimativa de esforço por analogia em contextos *cross-repository*.

5.1 Análise Exploratória dos Dados

Antes de proceder com a aplicação dos modelos de aprendizado de máquina, realizou-se uma análise exploratória detalhada do conjunto de dados para compreender suas características fundamentais, identificar padrões, *outliers* e peculiaridades que pudessem influenciar o desempenho preditivo dos modelos. Esta etapa é essencial para validar a qualidade dos dados,

justificar decisões metodológicas e contextualizar os resultados obtidos em relação à literatura existente (Fávero; Casanova; Pimentel, 2022).

O conjunto de dados analisado compreende 23.313 requisitos de software (*user stories*) distribuídos entre 16 projetos de código aberto. Cada requisito possui um valor associado de *story points*, que representa a estimativa de esforço de desenvolvimento atribuída pela equipe responsável. A análise exploratória foi conduzida em quatro dimensões principais: estatísticas descritivas gerais, distribuição de *story points*, heterogeneidade entre projetos e características textuais dos requisitos.

5.1.1 Estatísticas Descritivas e Distribuição de *Story Points*

A Tabela 2 apresenta as estatísticas descritivas do conjunto de dados, evidenciando características importantes para a modelagem. Observa-se que a distribuição de *story points* é fortemente assimétrica à direita (*right-skewed*), com média ($\mu = 6,22$) significativamente superior à mediana ($Md = 4,0$), indicando a presença de valores extremos que elevam a média. O desvio padrão elevado ($\sigma = 10,01$) e o coeficiente de assimetria muito alto (*skewness* = 6,00) confirmam esta distribuição altamente concentrada em valores baixos com cauda longa à direita. A curtose extremamente elevada (45,68) indica presença de valores extremos de (*outliers*) muito distantes da média, o que é esperado em projetos heterogêneos de desenvolvimento de software onde alguns requisitos excepcionalmente complexos podem receber estimativas muito superiores à maioria.

Tabela 2 – Estatísticas descritivas de *Story Points* do conjunto de dados.

Métrica	Valor
Total de amostras (<i>N</i>)	23.313
Número de projetos	16
Mínimo	1
Máximo	100
Média (μ)	6,22
Mediana (<i>Md</i>)	4,0
Moda	5,0
Desvio padrão (σ)	10,01
Primeiro quartil (Q_1)	2,0
Terceiro quartil (Q_3)	8,0
Amplitude interquartil (IQR)	6,0
Assimetria (<i>skewness</i>)	6,00
Curtose (<i>kurtosis</i>)	45,68

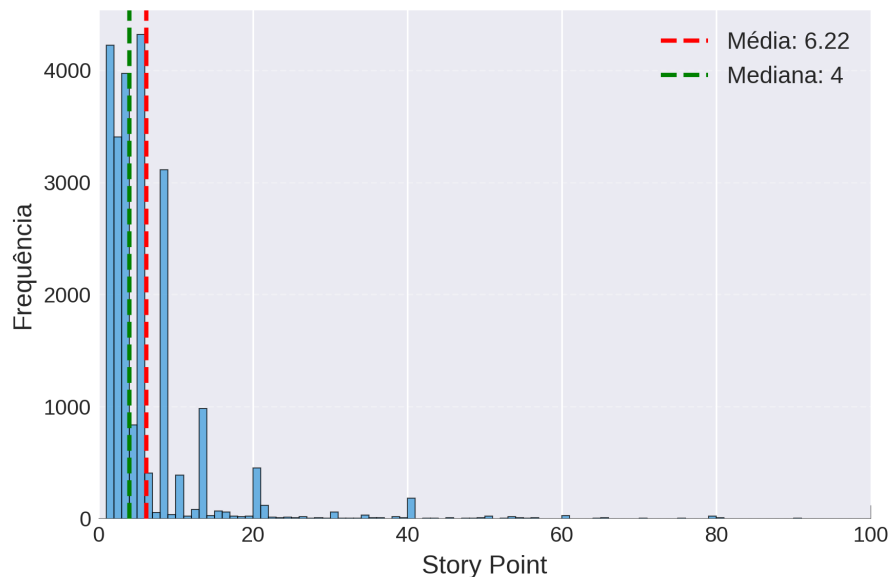
Fonte: Autoria própria (2025).

Observação importante: As três bases analisadas (*Base_Original*, *Corpus_1* e *Corpus_2*) apresentam estatísticas de *story points* idênticas, uma vez que o pré-processamento textual aplicado não alterou os valores de esforço atribuídos, apenas refinou o conteúdo textual dos requisitos. Portanto, as análises estatísticas relacionadas à distribuição

de *story points*, outliers, normalidade e heterogeneidade entre projetos são comuns às três bases, enquanto as características textuais diferem conforme o tipo de processamento aplicado.

A Figura 11 ilustra a distribuição de *story points* por meio de um histograma, confirmando a assimetria positiva e a concentração de valores nas classes inferiores. A moda igual a 5 *story points* e a mediana em 4 indicam que a maioria dos requisitos possui estimativas de esforço moderadas, embora a presença de valores extremos (máximo de 100 *story points*) eleve significativamente a média para 6,22. Esta distribuição é consistente com a literatura de engenharia de software, onde a maioria dos requisitos corresponde a tarefas de complexidade baixa a moderada, com uma minoria de requisitos excepcionalmente complexos (Choetkiertikul *et al.*, 2019).

Figura 11 – Histograma representando a medida de Story Point em relação à sua frequência no Corpus



Fonte: Autoria própria (2025)

5.1.2 Detecção de *Outliers*

Utilizando o método de detecção de *outliers* baseado na amplitude interquartil (IQR), identificou-se que 5,58% das amostras ($n = 1.302$) apresentam valores de *story points* considerados atípicos. A Tabela 3 apresenta os resultados da análise de *outliers*.

Os *outliers* identificados correspondem exclusivamente a requisitos com *story points* ≥ 18 , todos acima do limite superior calculado ($Q_3 + 1,5 \times \text{IQR} = 8 + 1,5 \times 6 = 17,0$). A presença desses valores extremos reflete requisitos de alta complexidade ou épicos¹ que foram

¹ No desenvolvimento ágil, *épico* é uma iniciativa de alto nível (ampla em escopo e valor de negócio) que excede a granularidade de uma *user story* (Cohn, 2004; Rubin, 2012). Estimar épicos como *stories* tende a inflar variância e gerar *outliers* nos *story points*.

Tabela 3 – Análise de outliers pelo método IQR

Métrica	Valor
Total de amostras (N)	23.313
Número de <i>outliers</i>	1.302
Percentual de <i>outliers</i>	5,58%
Limite inferior (IQR)	-7,0
Limite superior (IQR)	17,0
Valor mínimo dos <i>outliers</i>	18,0
Valor máximo dos <i>outliers</i>	100,0

Fonte: Autoria própria (2025).

incorretamente estimados na granularidade de *user stories*, sendo importante considerar sua influência na modelagem e avaliação do desempenho preditivo. Métodos robustos a *outliers*, como o erro absoluto mediano (MdAE), tornam-se relevantes neste contexto.

5.1.3 Teste de Normalidade

Para verificar a adequação de métricas de erro baseadas em distribuição normal (como o MSE), aplicou-se o teste de normalidade de *D'Agostino-Pearson* à distribuição de *story points*. A Tabela 4 apresenta os resultados do teste.

Tabela 4 – Teste de normalidade (*D'Agostino-Pearson*)

Base	Estatística	p-valor	Distribuição Normal?
Base Original / Corpus 1 / Corpus 2	26.335,13	< 0,001	Não

Fonte: Autoria própria (2025).

O teste de normalidade rejeita fortemente a hipótese nula de distribuição normal ($p < 0,001$), com estatística de teste extremamente elevada (26.335,13). Este resultado confirma que a distribuição de *story points* apresenta desvios significativos da normalidade, principalmente devido à assimetria acentuada (6,00) e curtose elevada (45,68).

Implicação metodológica: A distribuição não-normal justifica a utilização de métricas robustas como o erro absoluto médio (MAE) e o erro absoluto mediano (MdAE) em detrimento do erro quadrático médio (MSE), uma vez que este último é altamente sensível a *outliers* e assume distribuição gaussiana dos resíduos (Willmott; Matsuura, 2005).

5.1.4 Heterogeneidade entre Projetos

A Tabela 5 apresenta estatísticas descritivas separadas por projeto, evidenciando heterogeneidade significativa entre os repositórios. A média de *story points* varia consideravelmente entre projetos (de 2,13 em DuraCloud até 15,54 em Moodle), assim como o desvio padrão apresenta variabilidade substancial (de 1,40 em Usergrid até 21,65 em Moodle). Esta variabilidade interprojetos é atribuída a diferenças em práticas de estimativa, composição das equipes,

complexidade dos domínios de aplicação e critérios de granularidade dos requisitos (Fávero; Casanova; Pimentel, 2022).

Tabela 5 – Estatísticas descritivas de *story points* por projeto

ID	Projeto	N	Média	DP	Mediana	Mínimo	Máximo	IQR
AS	Appcelerator Studio	2919	5,64	3,33	5,0	1	40	5,0
AP	Aptanastudio	829	8,02	5,95	8,0	1	40	8,0
BB	Bamboo	521	2,42	2,14	2,0	1	20	2,0
CV	Clover	384	4,59	6,55	2,0	1	40	4,0
DM	Data Management	4667	9,57	16,60	4,0	1	100	7,0
DC	DuraCloud	666	2,13	2,03	1,0	1	16	1,0
JI	JIRA Software	352	4,43	3,51	3,0	1	20	3,0
ME	Mesos	1680	3,09	2,42	3,0	1	40	1,0
MD	Moodle	1166	15,54	21,65	8,0	1	100	17,0
MU	Mule	889	5,08	3,50	5,0	1	21	5,0
MS	Mule Studio	732	6,40	5,39	5,0	1	34	5,0
XD	Spring XD	3526	3,70	3,23	3,0	1	40	3,0
TD	Talend Data Quality	1381	5,92	5,19	5,0	1	40	6,0
TE	Talend ESB	868	2,16	1,50	2,0	1	13	2,0
TI	Titanium	2251	6,32	5,10	5,0	1	34	5,0
UG	Usergrid	482	2,85	1,40	3,0	1	8	1,0
Geral (todos os projetos)		23313	6,22	10,01	4,0	1	100	6,0

Fonte: Autoria própria (2025).

Observa-se que projetos como *Moodle* (média = 15,54, DP = 21,65) e *Data Management* (média = 9,57, DP = 16,60) apresentam estimativas substancialmente superiores à média geral (6,22), com alta dispersão indicando requisitos de complexidade muito heterogênea. Em contraste, projetos como *DuraCloud* (média = 2,13, DP = 2,03), *Talend ESB* (média = 2,16, DP = 1,50) e *Bamboo* (média = 2,42, DP = 2,14) concentram-se em estimativas baixas com variabilidade reduzida, sugerindo requisitos de granularidade mais uniforme.

A Tabela 6 apresenta os resultados do teste ANOVA aplicado para verificar estatisticamente a heterogeneidade entre projetos.

Tabela 6 – Teste ANOVA para heterogeneidade entre projetos

Base	F-statistic	p-valor	Heterogêneo?
Base Original / Corpus 1 / Corpus 2	176,23	< 0,001	Sim

Fonte: Autoria própria (2025).

A análise de variância (ANOVA) confirma que as diferenças observadas entre projetos são estatisticamente muito significativas ($F = 176,23$, $p < 0,001$), justificando a adoção de metodologias de validação *cross-repository* como o *LOPO*. O valor elevado da estatística F indica que a variância entre grupos (projetos) é substancialmente maior que a variância intragrupos, demonstrando que cada projeto possui características de estimativa distintas e não generalizáveis trivialmente.

Implicação metodológica: Esta heterogeneidade representa um desafio fundamental para modelos de estimativa de esforço, uma vez que características específicas de cada projeto podem influenciar padrões de complexidade e estimativas de esforço de maneira não generalizável. O teste ANOVA justifica a escolha da metodologia *Leave-One-Project-Out (LOPO)* para o Experimento E2, garantindo avaliação da capacidade de generalização do modelo para novos projetos.

5.1.5 Análise Textual dos Requisitos

A análise das características textuais dos requisitos revela informações relevantes para a compreensão da qualidade e representatividade semântica de cada *corpus*. Diferentemente das estatísticas de *story points*, que são idênticas nas três bases, as métricas textuais diferem significativamente conforme o tipo de pré-processamento aplicado. A Tabela 7 apresenta as estatísticas relacionadas ao comprimento dos requisitos (em número de palavras) e à correlação com *story points*.

Tabela 7 – Estatísticas da análise textual dos requisitos

Base	Palavras Média	Palavras Mediana	Palavras Desvio	Palavras Máximo	Corr. Palavras-SP	Corr. Caracteres-SP
Base Original	77,08	49,0	129,37	3.026	−0,007	−0,033
Corpus_1	73,03	47,0	125,01	2.532	−0,003	−0,007
Corpus_2	29,59	29,0	8,82	157	+0,041	+0,048

Fonte: Autoria própria (2025).

Os resultados evidenciam diferenças substanciais entre os *corpora*:

- **Base Original:** Apresenta requisitos extensos (média de 77 palavras) com alta variabilidade ($\sigma = 129,37$) e valores extremos (até 3.026 palavras), indicando presença de descrições detalhadas, textos duplicados ou formatação inconsistente.
- **Corpus_1:** O pré-processamento tradicional reduziu ligeiramente o comprimento médio (73 palavras) e a variabilidade ($\sigma = 125,01$), removendo ruídos textuais, mas mantendo uma estrutura similar à base original.
- **Corpus_2:** O refinamento via *GPT-5 mini* produziu requisitos significativamente mais concisos (média de 29,59 palavras), com baixa variabilidade ($\sigma = 8,82$) e comprimentos máximos controlados (157 palavras). Esta padronização reflete a expansão de abreviações, correção gramatical e reformulação estruturada aplicada pelo modelo de linguagem.

5.1.5.1 Correlação entre Comprimento Textual e *Story Points*

A análise de correlação revela padrões interessantes:

- **Base Original e Corpus_1:** Apresentam correlação negativa fraca entre comprimento textual e *story points* (Pearson $\rho \approx -0,007$ e $-0,003$, respectivamente), indicando ausência de relação linear consistente. Requisitos mais longos não estão sistematicamente associados a estimativas de esforço superiores, sugerindo que o tamanho da descrição não reflete, por si só, a complexidade técnica.
- **Corpus_2 (GPT):** Apresenta correlação positiva fraca, porém não desprezível ($\rho = +0,041$ para palavras, $+0,048$ para caracteres). Este resultado sugere que o refinamento via *GPT-5 mini*, ao padronizar e estruturar os requisitos, introduziu uma relação mais consistente entre tamanho textual e complexidade estimada, possivelmente por expandir descrições de requisitos complexos e condensar descrições redundantes.

Implicação para modelagem: A correlação consistentemente fraca ($|\rho| < 0,05$) entre comprimento textual e *story points* reforça a necessidade de abordagens baseadas em representações semânticas profundas, como as proporcionadas por modelos de linguagem pré-treinados (BERT) e propagação estrutural via GCN. Isso significa que características superficiais como comprimento textual são insuficientes para capturar a complexidade inerente aos requisitos de software, validando a escolha da arquitetura BertGCN que opera sobre *embeddings* contextualizados ao invés de *features* léxicas simples.

5.2 Pré-processamento dos Dados

Conforme descrito na Seção 4.2.2 do Capítulo 4, foram aplicadas duas estratégias de pré-processamento gerando os *corpora* Corpus_1 (tradicional) e Corpus_2 (refinado via *GPT-5 mini*). A Tabela 8 apresenta um exemplo representativo das transformações aplicadas em cada etapa do processamento, desde o requisito original até as versões processadas em ambos os *corpora*.

O exemplo ilustra três níveis distintos de processamento textual:

- **Requisito Original:** Contém marcações HTML (`{html}`, `<div>`, `<p>`), formatação inconsistente, abreviações técnicas (iOS) e estrutura narrativa fragmentada típica de sistemas de rastreamento de *issues*.
- **Corpus_1 (Pré-processamento Tradicional):** Aplica limpeza textual removendo marcações HTML, convertendo para minúsculas, eliminando pontuação e normalizando espaços. Preserva a estrutura original do texto, mas remove ruídos sintáticos. O resultado mantém termos técnicos e estrutura imperativa.

Tabela 8 – Exemplo de transformações aplicadas no pré-processamento

Requisito Original	Create packager for Mobile/iOS project {html} Follow existing screens/workflow as appropriate. Launch wizard from right-click menu in App Explorer and button on App Explorer top bar. {html}
Corpus_1	create packager for mobile project follow existing screens as appropriate launch wizard from right click menu in app explorer and button on app explorer top bar
Corpus_2	As a user I want a mobile project packager that follows existing screens and launches a wizard from the App Explorer context menu and the App Explorer toolbar.

Fonte: Autoria própria (2025).

- **Corpus_2 (Refinamento via GPT-5 mini):** Reestrutura completamente o requisito seguindo o formato canônico de *user story* (“As a [role] I want [feature] so that [benefit]”), expande abreviações implícitas (“packager” → “mobile project packager”), normaliza terminologia técnica (“right-click menu” → “context menu”, “button on top bar” → “toolbar”) e estabelece coesão narrativa entre os elementos funcionais.

Estas transformações refletem abordagens distintas de pré-processamento: o *Corpus_1* foca em limpeza e normalização lexical preservando a estrutura original, enquanto o *Corpus_2* realiza reestruturação semântica profunda visando padronização narrativa e expansão de contexto implícito, potencialmente aprimorando a qualidade das representações semânticas extraídas pelo modelo BERT.

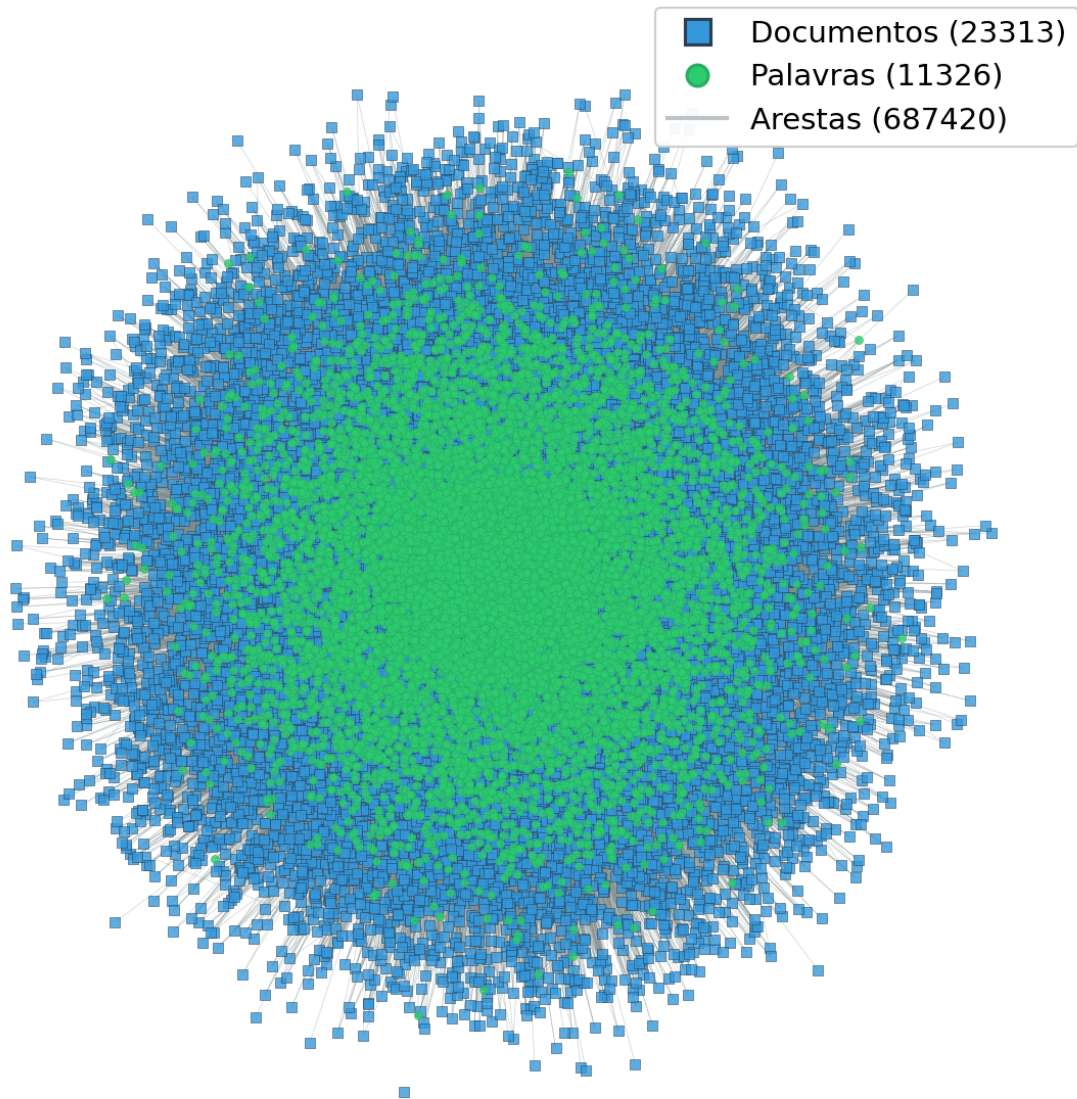
5.3 Estrutura do Grafo Heterogêneo

O modelo BertGCN fundamenta-se na construção de um grafo heterogêneo bipartido que captura relações semânticas entre documentos (requisitos de software) e palavras (vocabulário). Esta estrutura de grafo constitui o elemento central da arquitetura, permitindo a propagação de informações contextuais através das camadas convolucionais de grafos (GCN) e possibilitando que requisitos semanticamente relacionados influenciem mutuamente suas representações, mesmo quando pertencentes a projetos distintos.

5.3.1 Visualização da Estrutura do Grafo

A Figura 12 apresenta a visualização da estrutura completa do grafo heterogêneo construído para o *Corpus_1*. Observa-se a presença de um nó isolado (grau zero), um documento fica sem arestas devido ao filtro de frequência mínima ($min_df = 3$) aplicado ao vocabulário, que remove todas as palavras de requisitos contendo apenas termos raros. Nós isolados não participam da propagação de informação via camadas GCN, limitando a capacidade preditiva do modelo para os requisitos afetados.

Figura 12 – Visualização do grafo heterogêneo construído para o Corpus_1. Grafo com 23.313 documentos, 11.326 palavras e 687.420 arestas



Fonte: Autoria própria (2025).

5.4 Resultados dos Experimentos

Nesta seção, são apresentados os resultados quantitativos obtidos a partir da aplicação do modelo BertGCN para estimativa de esforço de software. Os experimentos foram conduzidos utilizando validação cruzada *nested k-fold* ($k_1 = 10$, $k_2 = 10$) para o cenário multi-repositórios (Experimento E1), e metodologia *LOPO* para o cenário *cross-repository* (Experimento E2), seguindo a metodologia proposta por Fávero, Casanova e Pimentel (2022). As métricas de avaliação utilizadas incluem o MAE, MSE e MdAE, permitindo análise abrangente do desempenho preditivo do modelo.

Todos os experimentos utilizaram o *corpus* completo contendo 23.313 requisitos de software distribuídos entre 16 projetos, construindo um grafo heterogêneo global conforme especificado pela arquitetura BertGCN original (Lin *et al.*, 2022). Os experimentos foram conduzidos para ambos os *corpora*: *Corpus_1* (pré-processamento tradicional) e *Corpus_2* (refinado via *GPT-5 mini*), permitindo avaliar o impacto do processamento avançado via LLM na qualidade das estimativas.

5.4.1 Experimento E1: Multi-repositórios

O Experimento E1 avaliou o desempenho do modelo BertGCN em cenário multi-repositórios, onde os conjuntos de treino e teste contêm requisitos provenientes de múltiplos projetos. Utilizando validação cruzada *nested k-fold* com $k_1 = 10$ *folds* externos e $k_2 = 10$ *folds* internos, este experimento simula um ambiente onde o modelo é treinado com dados heterogêneos e avaliado em amostras de diversos domínios.

A Tabela 9 apresenta os resultados consolidados do Experimento E1 para ambos os *corpora*, incluindo médias e desvios padrão das métricas calculadas sobre os 10 *folds* de validação.

Tabela 9 – Resultados do Experimento E1 (Multi-repositórios) - *Corpus_1* e *Corpus_2*

Corpus	MAE	MSE	MdAE
Corpus_1 (Tradicional)	$4,41 \pm 0,35$	$84,88 \pm 13,03$	$2,52 \pm 0,43$
Corpus_2 (Refinado <i>GPT-5 mini</i>)	$4,52 \pm 0,41$	$88,41 \pm 12,71$	$2,75 \pm 0,57$

Fonte: Autoria própria (2025).

Os resultados indicam desempenho comparável entre os dois *corpora*, com o *Corpus_1* apresentando ligeira vantagem em todas as métricas. O MAE médio de 4,41 *story points* para o *Corpus_1* e 4,52 para o *Corpus_2* demonstra que o refinamento via *GPT-5 mini* não resultou em melhorias significativas no desempenho preditivo, contrariando a hipótese inicial de que textos mais estruturados e gramaticalmente corretos aprimorariam as representações semânticas. Observa-se que o desvio padrão foi relativamente baixo para os valores de MAE e MdAE e para os valores de MSE foi equilibrado, o que reforça o afirmativo anterior.

5.4.2 Experimento E2: *Cross-repository*

O Experimento E2 avaliou a capacidade de generalização do modelo BertGCN para projetos completamente novos não vistos durante o treinamento, utilizando a metodologia *LOPO*. Neste cenário, cada um dos 16 projetos foi utilizado uma vez como conjunto de teste, enquanto os demais 15 projetos serviram para treinamento e validação interna ($k_2 = 10$ *folds*).

A Tabela 10 apresenta os resultados consolidados do Experimento E2 para ambos os *corpora*.

Tabela 10 – Resultados do Experimento E2 (Cross-repository LOPO) - Corpus_1 e Corpus_2

Corpus	MAE	MSE	MdAE
Corpus_1	4,46 ± 2,71	75,44 ± 123,56	3,09 ± 1,06
Corpus_2	4,27 ± 2,62	75,00 ± 121,89	2,88 ± 1,04

Fonte: Autoria própria (2025).

O Experimento E2 apresenta maior variabilidade (desvios padrão superiores) em comparação ao E1, refletindo a heterogeneidade entre projetos identificada na análise exploratória. Interessantemente, o *Corpus_2* demonstra ligeira vantagem no cenário *cross-repository*, com MAE médio de 4,27 contra 4,46 do *Corpus_1*, sugerindo que o refinamento via *GPT-5 mini* pode auxiliar na generalização entre domínios ao padronizar a estrutura narrativa dos requisitos.

A Tabela 11 apresenta os resultados detalhados do Experimento E2 para todos os 16 projetos, comparando o desempenho dos modelos BertGCN (com *Corpus_1* e *Corpus_2*) com os modelos de referência SE³M (Fávero; Casanova; Pimentel, 2022) e Z-SE² (Baratto, 2022). São apresentados o identificador do projeto (ID), o número de requisitos por projeto (Requisitos/projeto), a média (M_E) e o desvio padrão (DP) do esforço por requisito em cada projeto, e os valores de MAE obtidos pelos diferentes modelos. Os menores valores de MAE de cada projeto foram destacados em negrito.

Tabela 11 – Valores do MAE obtidos em cada subconjunto no experimento E2 dos modelos BertGCN (Corpus_1 e Corpus_2) e dos experimentos análogos realizados com SE³M e Z-SE², com dados dos respectivos projetos utilizados para avaliação. Também são apresentados o identificador do projeto (ID), o número de requisitos por projeto (Requisitos/projeto), a média (M_E) e o desvio padrão (DP) do esforço por requisito em cada projeto. Quanto menor o valor do MAE, melhor o resultado. Os menores valores de MAE de cada projeto foram destacados em negrito.

ID	Descrição	Requisitos/projeto	M_E	DP	MAE (Z-SE ²)	MAE (SE ³ M)	MAE (BertGCN-C1)	MAE (BertGCN-C2)
AS	Appcelerator Studio	2919	5,64	3,33	2,26	2,50	2,99	2,60
AP	Aptanastudio	829	8,02	5,95	4,05	4,18	4,33	4,52
BB	Bamboo	521	2,42	2,14	2,31	2,76	1,71	2,22
CV	Clover	384	4,59	6,55	3,54	3,87	4,27	3,92
DM	Data Management	4667	9,57	16,60	7,75	7,78	8,58	7,65
DC	DuraCloud	666	2,13	2,03	3,53	3,79	5,23	3,16
Jl	JIRA Software	352	4,43	3,51	4,75	3,13	6,64	2,86
ME	Mesos	1680	3,09	2,42	1,54	3,39	3,27	1,86
MD	Moodle	1166	15,54	21,65	11,54	11,99	12,08	12,54
MU	Mule	889	5,08	3,50	2,65	3,51	3,47	2,96
MS	Mule Studio	732	6,40	5,39	3,42	3,51	3,74	3,72
XD	Spring XD	3526	3,70	3,23	2,27	3,16	2,23	2,32
TD	Talend Data Quality	1381	5,92	5,19	3,54	4,04	4,19	4,42
TE	Talend ESB	868	2,16	1,50	2,67	3,42	1,86	5,16
TI	Titanium	2251	6,32	5,10	3,24	3,49	4,67	4,02
UG	Usergrid	482	2,85	1,40	1,76	3,24	2,18	4,32

Fonte: Adaptado de Fávero, Casanova e Pimentel (2022) e Baratto (2022) .

A análise comparativa revela desempenho heterogêneo entre projetos e modelos. O Z-SE² apresenta os melhores resultados em 10 dos 16 projetos (Appcelerator Studio, Aptanastudio, Clover, Mesos, Moodle, Mule, Mule Studio, Talend Data Quality, Titanium e Usergrid), consolidando-se como o modelo mais preciso no cenário *cross-repository*. O BertGCN-

Corpus_1 supera os demais em 3 projetos (Bamboo, Spring XD e Talend ESB), enquanto o BertGCN-Corpus_2 também lidera em 3 projetos (Data Management, DuraCloud e JIRA Software), demonstrando desempenhos equivalentes, mas em contextos distintos. O modelo SE³M, embora não lidere isoladamente em nenhum projeto, mantém resultados consistentemente competitivos, posicionando-se frequentemente em segundo lugar.

Destaques de desempenho:

- **Z-SE²**: Melhor desempenho geral, liderando em 10 projetos (62,5% do total) com resultados excepcionais em Mesos (MAE = 1,54), Usergrid (MAE = 1,76), Appcelerator Studio (MAE = 2,26) e Mule (MAE = 2,65). Apresenta superioridade consistente especialmente em projetos com alta variabilidade, como Moodle (MAE = 11,54 vs. 11,99–12,54 dos demais modelos) e Titanium (MAE = 3,24 vs. 3,49–4,67).
- **BertGCN-Corpus_1**: Alcança os melhores resultados em 3 projetos com características distintas: Bamboo (MAE = 1,71), Spring XD (MAE = 2,23) e Talend ESB (MAE = 1,86). Estes projetos compartilham distribuições relativamente regulares e baixa variabilidade (Bamboo: DP = 2,14; Talend ESB: DP = 1,50; Spring XD: DP = 3,23), sugerindo que o pré-processamento tradicional preserva características textuais informativas em contextos menos heterogêneos.
- **BertGCN-Corpus_2**: Demonstra vantagens significativas em outros 3 projetos: JIRA Software (MAE = 2,86 vs. 3,13–6,64), DuraCloud (MAE = 3,16 vs. 3,53–5,23) e Data Management (MAE = 7,65 vs. 7,75–8,58). Interessantemente, estes projetos apresentam heterogeneidade interna elevada (Data Management: DP = 16,60), sugerindo que o refinamento via GPT-5 mini auxilia especialmente em contextos com alta variabilidade e possível presença de termos ou abreviações inconsistentes.
- **SE³M**: Embora não lidere em nenhum projeto isoladamente, mantém desempenho consistente e competitivo em todos os 16 projetos, posicionando-se frequentemente em segundo lugar (ex., JIRA Software: MAE = 3,13; Titanium: MAE = 3,49), demonstrando estabilidade preditiva em cenários *cross-repository*.

Este resultado evidencia a dificuldade de transferência de conhecimento entre projetos com práticas de estimativa heterogêneas. Projetos com distribuições atípicas apresentam erros elevados em todos os modelos: *Moodle* (média = 15,54 *story points*, DP = 21,65) registra MAE entre 11,54 e 12,54, enquanto *Data Management* (média = 9,57, DP = 16,60) apresenta MAE entre 7,65 e 8,58. Em contraste, projetos com distribuições regulares como *Bamboo* (média = 2,42, DP = 2,14), *DuraCloud* (média = 2,13, DP = 2,03) e *Talend ESB* (média = 2,16, DP = 1,50) permitem erros de predição substancialmente menores (MAE entre 1,71 e 3,16).

Comparando as duas variantes BertGCN, observa-se especialização complementar: Corpus_2 supera Corpus_1 em 9 dos 16 projetos, com reduções expressivas de MAE em

JIRA Software ($-56,9\%$), Mesos ($-43,1\%$) e DuraCloud ($-39,6\%$). Inversamente, *Corpus_1* obtém desempenho superior em 7 projetos, notadamente em Talend ESB (1,86 vs. 5,16; $+177,4\%$) e Usergrid (2,18 vs. 4,32; $+98,2\%$). Esses resultados sugerem que o refinamento textual do *Corpus_2* favorece cenários heterogêneos, enquanto a preservação de termos técnicos originais do *Corpus_1* captura informação semântica relevante em domínios específicos.

5.4.3 Análise Comparativa com a Literatura

A Tabela 12 compara o desempenho do modelo BertGCN proposto neste trabalho com os resultados reportados por Baratto (2022) e Fávero, Casanova e Pimentel (2022) para os modelos Z-SE² e SE³M, que aplicaram a mesma base de dados em seus experimentos.

Tabela 12 – Erro Absoluto Médio (MAE), Erro Quadrático Médio (MSE) e Mediana dos Erros Absolutos (MdAE) obtidos ao aplicar o BertGCN, comparados aos modelos SE³M, Z-SE² e Deep-SE. Para todas as métricas, quanto menor o valor melhor o resultados.

Modelo	Experimento	MAE	MSE	MdAE
Z-SE ² (Baratto, 2022)	E1	$3,67 \pm 0,12$	64,38	1,88
SE ³ M (Fávero et al., 2022)	E1	$4,25 \pm 0,17$	86,15	2,30
Deep-SE (Choetkiertikul et al., 2019)	E2	$3,82 \pm 1,56$	—	—
BertGCN - Corpus_1	E1	$4,41 \pm 0,35$	84,88	2,52
BertGCN - Corpus_2	E1	$4,52 \pm 0,41$	88,41	2,75
BertGCN - Corpus_1	E2	$4,46 \pm 2,71$	75,44	3,09
BertGCN - Corpus_2	E2	$4,27 \pm 2,62$	75,00	2,88

Fonte: Autoria própria (2025).

No cenário multi-repositórios (E1), o modelo BertGCN apresenta desempenho comparável ao SE³M, porém inferior ao Z-SE², que alcança o melhor desempenho geral (MAE = 3,67). A diferença de aproximadamente 0,74 *story points* entre o Z-SE² e o BertGCN-Corpus_1 sugere que a arquitetura e as técnicas empregadas por Baratto (2022) resultam em estimativas mais precisas.

No cenário *cross-repository* (E2), o modelo BertGCN apresenta desempenho inferior ao Deep-SE (4,27 vs 3,82 no MAE), embora a alta variabilidade observada ($\pm 2,62$) torne as comparações menos conclusivas. Respondendo a questão central do trabalho, os resultados sugerem que a propagação estrutural via GCN, embora teoricamente capaz de capturar relações semânticas entre projetos, não oferece vantagens decisivas em cenários de alta heterogeneidade inter-repositórios.

5.4.4 Discussão dos Resultados

Os resultados obtidos neste trabalho levantam reflexões importantes sobre a eficácia de arquiteturas baseadas em grafos heterogêneos para estimativa de esforço de software e sobre o papel do refinamento textual via modelos de linguagem de grande escala.

5.4.4.1 Desempenho do BertGCN em Relação aos Modelos SE³M e Z-SE²

A performance do modelo BertGCN em ambos os experimentos situa-se próxima aos modelos da literatura, porém sem superá-los de forma consistente. O Z-SE² de Baratto (2022) mantém-se como o modelo mais preciso no cenário E1, com vantagem de aproximadamente 17% em relação ao BertGCN-Corpus_1 (3,67 vs 4,41 no MAE). Duas hipóteses podem explicar esta diferença:

1. **Limitações do aprendizado transdutivo:** A abordagem transdutiva, embora teoricamente vantajosa para capturar estruturas globais, pode introduzir viés quando aplicada à validação cruzada, uma vez que todos os documentos (incluindo os de teste) participam da construção do grafo. Modelos indutivos como Z-SE² e SE³M treinam exclusivamente em dados de treino, resultando em estimativas potencialmente mais realistas.
2. **Informação estrutural limitada:** Embora o grafo heterogêneo capture co-ocorrências entre palavras e documentos, a estrutura resultante pode ser esparsa para projetos com vocabulários distintos, limitando o ganho de informação proporcionado pela propagação via GCN.

Em relação direta ao SE³M de Fávero, Casanova e Pimentel (2022), observa-se que, no cenário E1, o BertGCN-Corpus_1 apresenta desempenho ligeiramente inferior: seu MAE ($4,41 \pm 0,35$) é cerca de 3,8% maior que o do SE³M ($4,25 \pm 0,17$), enquanto o BertGCN-Corpus_2 afasta-se ainda mais, com MAE de $4,52 \pm 0,41$ (aproximadamente 6,4% acima do SE³M). Tendência similar é observada na mediana do erro absoluto (MdAE), na qual o SE³M mantém valores inferiores aos obtidos pelas duas variantes do BertGCN. Esses resultados indicam que, embora o BertGCN apresente desempenho competitivo e dentro da mesma ordem de grandeza, o SE³M permanece como *baseline* ligeiramente superior no cenário avaliado.

5.4.4.2 Impacto do Refinamento via GPT-5 mini

Contrariando a hipótese inicial, o refinamento textual via GPT-5 mini (Corpus_2) não resultou em melhorias significativas no desempenho preditivo. No Experimento E1, o Corpus_2 apresentou desempenho ligeiramente inferior ao Corpus_1 (4,52 vs 4,41 no MAE), enquanto no Experimento E2 observou-se ligeira vantagem (4,27 vs 4,46).

Dois fatores podem explicar estes resultados:

1. **Preservação de termos técnicos:** O modelo BERT pré-treinado foi exposto a vocabulário técnico de domínios variados durante seu treinamento em larga escala. A expansão de abreviações e normalização gramatical realizadas pelo *GPT-5 mini* pode ter removido sinais informativos implícitos em termos específicos de cada projeto, reduzindo a capacidade do modelo de distinguir entre contextos específicos distintos (entre projetos).
2. **Padronização excessiva:** O refinamento via LLM tende a padronizar a estrutura narrativa dos requisitos, potencialmente reduzindo a variabilidade textual que poderia auxiliar na diferenciação entre requisitos de complexidades distintas. Em tarefas de regressão, a preservação de características próprias do domínio pode ser mais valiosa que a correção gramatical.

5.4.4.3 Heterogeneidade e Generalização *Cross-Repository*

A alta variabilidade observada no Experimento E2 ($MAE = 4,46 \pm 2,71$ para *Corpus_1*) confirma a heterogeneidade significativa entre projetos identificada na análise exploratória. Projetos como *Moodle* ($MAE = 12,08$) e *Data Management* ($MAE = 8,58$) apresentam erros substancialmente superiores à média, enquanto projetos como *Bamboo* ($MAE = 1,71$) e *Mesos* ($MAE = 3,27$) demonstram estimativas relativamente precisas.

Estes resultados reforçam a conclusão de Fávero, Casanova e Pimentel (2022) de que a generalização *cross-repository* permanece um desafio fundamental em estimativa de esforço por analogia. Práticas de estimativa, critérios de granularidade e contextos organizacionais variam substancialmente entre projetos, limitando a transferência de conhecimento mesmo quando se empregam representações semânticas profundas.

5.4.4.4 Limitações Metodológicas e Trabalhos Futuros

Este trabalho apresenta limitações metodológicas que devem ser consideradas na interpretação dos resultados:

1. **Aprendizado transdutivo vs. indutivo:** A construção de um grafo global contendo todos os documentos, embora fiel à arquitetura BertGCN original, não reflete cenários reais de produção onde novos projetos surgem sem acesso ao grafo de treinamento. Versões indutivas do modelo, capazes de inferir estimativas para grafos não vistos durante o treinamento, constituem direção promissora para trabalhos futuros.
2. **Exploração limitada do espaço de hiperparâmetros:** O grid search empregado considerou quatro configurações principais. Técnicas de otimização bayesiana ou busca

aleatória mais extensiva poderiam identificar combinações de hiperparâmetros mais eficazes.

3. **Engenharia de *features* complementares:** A arquitetura BertGCN utiliza exclusivamente informações textuais. A incorporação de metadados de projetos (tamanho da equipe, experiência, domínio de aplicação) como *node features* adicionais poderia aprimorar a capacidade preditiva do modelo.
4. **Análise de grafos alternativos:** Este trabalho seguiu a construção de grafos heterogêneos proposta por Lin *et al.* (2022). Abordagens alternativas, como grafos baseados em similaridade semântica de *embeddings* BERT ou grafos temporais capturando evolução de requisitos, poderiam oferecer representações estruturais mais informativas.
5. **Validação em bases adicionais de larga escala:** Os experimentos foram conduzidos exclusivamente sobre o conjunto de 16 projetos de Choetkiertikul *et al.* (2019). A replicação desta investigação em bases maiores e mais heterogêneas de issues ágeis, como os corpora multiprojeto analisados por Tawosi, Moussa e Sarro (2022), permitiria verificar se os resultados observados neste estudo se mantêm em cenários mais amplos e variados.

Os resultados obtidos contribuem para o corpo de conhecimento em estimativa de esforço de software ao demonstrar empiricamente as limitações de arquiteturas baseadas em grafos heterogêneos em cenários de alta heterogeneidade inter-repositórios, reforçando a necessidade de abordagens que integrem múltiplas fontes de informação além do conteúdo textual dos requisitos.

6 CONCLUSÃO

Este trabalho avaliou a aplicação do modelo BertGCN para estimativa de esforço em software, utilizando um corpus de 23.313 requisitos de 16 projetos. Os experimentos demonstraram que o modelo, empregando aprendizado transdutivo e grafos heterogêneos com arestas TF-IDF e PPMI, alcançou desempenho geral inferior quando comparado a modelos como SE³M e Z-SE², com MAE de $4,41 \pm 0,35$ no cenário multi-repositório e $4,46 \pm 2,71$ no cenário *cross-repository* (LOPO).

A análise exploratória dos dados revelou uma distribuição não-normal dos *story points* (assimetria=6,00; curtose=45,68), com significativa heterogeneidade entre projetos (ANOVA: $F=176,23$, $p<0,001$) e presença de 5,58% de outliers. Essas características evidenciam a complexidade inerente à tarefa de estimativa e justificam a necessidade de modelos robustos capazes de lidar com variabilidade inter-projetos.

A partir dos resultados, algumas limitações foram identificadas. O paradigma transdutivo do BertGCN requer acesso a todos os dados durante o treinamento, limitando sua aplicabilidade em cenários de produção com novos requisitos não vistos. Além disso, a exploração de hiperparâmetros foi restrita a valores pré-definidos, sem busca sistemática para otimização no domínio específico de estimativa de esforço. O modelo também não incorporou *features* complementares como métricas de código ou histórico de equipe, que poderiam enriquecer as representações.

As principais contribuições deste trabalho incluem: (1) a adaptação e avaliação do BertGCN para estimativa de story points, domínio ainda pouco explorado na literatura; (2) a realização de experimentos sistemáticos em dois cenários complementares (multi-repositório e cross-repositório) utilizando validação cruzada aninhada e LOPO; (3) a análise estatística abrangente do corpus, documentando suas características e desafios; e (4) a comparação com modelos estabelecidos, fornecendo *baseline* para trabalhos futuros.

REFERÊNCIAS

- ABDUKALYKOV, R. **A New Methodology for Quantifying the Impact of Non-Functional Requirements on Software Effort Estimation**. 2011. Tese (Doutorado) — Concordia University, 2011.
- AGARAP, A. F. Deep learning using rectified linear units (relu). **arXiv preprint arXiv:1803.08375**, 2018.
- ALAMMAR, J. Blog, **The Illustrated Transformer**. 2018. Disponível em: <https://jalammar.github.io/illustrated-transformer/>.
- BACKSTROM, L.; SUN, E.; MARLOW, C. Find me if you can: improving geographical prediction with social and spatial proximity. *In: Proceedings of the 19th international conference on World wide web*. [S.l.: s.n.], 2010. p. 61–70.
- BARATTO, G. J. **Comparação entre os modelos pré-treinados GPT-3 e BERT na estimativa de esforço de software por analogia a partir de requisitos textuais**. 2022. Dissertação (B.S. thesis) — Universidade Tecnológica Federal do Paraná, 2022.
- BATTAGLIA, P. W. *et al.* Relational inductive biases, deep learning, and graph networks. **arXiv preprint arXiv:1806.01261**, 2018.
- BELKIN, M.; NIYOGI, P.; SINDHWANI, V. Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. **Journal of Machine Learning Research**, v. 7, p. 2399–2434, 2006. Disponível em: <https://www.jmlr.org/papers/volume7/belkin06a/belkin06a.pdf>.
- BENGIO, Y.; COURVILLE, A.; VINCENT, P. Representation learning: A review and new perspectives. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, IEEE, v. 35, n. 8, p. 1798–1828, 2013. Disponível em: <https://doi.org/10.1109/TPAMI.2013.50>.
- BENGIO, Y.; COURVILLE, A.; VINCENT, P. Representation learning: A review and new perspectives. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 35, n. 8, p. 1798–1828, 2013.
- BOEHM, B.; ABTS, C.; CHULANI, S. Software development cost estimation approaches—a survey. **Annals of software engineering**, Springer, v. 10, n. 1-4, p. 177–205, 2000. Disponível em: <https://doi.org/10.1023/A:1018991717352>.
- BREIMAN, L. Bagging predictors. **Machine Learning**, v. 24, n. 2, p. 123–140, 1996.
- BROWN, T. B. *et al.* Language models are few-shot learners. *In: Proceedings of the 34th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2020. (NIPS'20). ISBN 9781713829546. Disponível em: <https://dl.acm.org/doi/abs/10.5555/3495724.3495883>.
- CAI, H.; ZHENG, V. W.; CHANG, K. C.-C. A comprehensive survey of graph embedding: Problems, techniques, and applications. **IEEE transactions on knowledge and data engineering**, IEEE, v. 30, n. 9, p. 1616–1637, 2018.
- CAWLEY, G. C.; TALBOT, N. L. On over-fitting in model selection and subsequent selection bias in performance evaluation. **Journal of Machine Learning Research**, v. 11, p. 2079–2107, 2010.

- CHAI, T.; DRAXLER, R. R. Root mean square error (rmse) or mean absolute error (mae)? – arguments against avoiding rmse in the literature. **Geoscientific Model Development**, v. 7, p. 1247–1250, 2014.
- CHAPELLE, O.; SCHÖLKOPF, B.; ZIEN, A. (Ed.). **Semi-Supervised Learning**. Cambridge, MA: MIT Press, 2006.
- CHIU, N.-H.; HUANG, S.-J. The adjusted analogy-based software effort estimation based on similarity distances. **Journal of Systems and Software**, v. 80, n. 4, p. 628–640, abr. 2007. ISSN 0164-1212. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0164121206001701>.
- CHOETKIERTIKUL, M. *et al.* A Deep Learning Model for Estimating Story Points. **IEEE Transactions on Software Engineering**, v. 45, n. 7, p. 637–656, jul. 2019. ISSN 1939-3520. Disponível em: <https://doi.org/10.1109/TSE.2018.2792473>.
- CHOI, S.; PARK, S.; SUGUMARAN, V. A rule-based approach for estimating software development cost using function point and goal and scenario based requirements. **Expert Systems with Applications**, Elsevier, v. 39, n. 1, p. 406–418, 2012.
- COHEN, J. **Statistical Power Analysis for the Behavioral Sciences**. 2. ed. New York: Routledge, 2009.
- COHN, M. **User Stories Applied: For Agile Software Development**. Boston: Addison-Wesley, 2004. ISBN 978-0321205681.
- COHN, M. **Agile Estimating and Planning**. USA: Prentice Hall PTR, 2005. ISBN 0131479415. Disponível em: <https://dl.acm.org/doi/10.5555/1036751>.
- D'AGOSTINO, R. B.; PEARSON, E. S. Tests for departure from normality. empirical results for the distributions of b_2 and $\sqrt{b_1}$. **Biometrika**, v. 60, n. 3, p. 613–622, 1973.
- DEVLIN, J. *et al.* BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. **CoRR**, abs/1810.04805, 2018. Disponível em: <http://arxiv.org/abs/1810.04805>.
- DIESTEL, R. **Graph Theory**. 5. ed. [S.l.]: Springer, 2017. (Graduate Texts in Mathematics).
- FARZINDAR, A.; INKPEN, D.; HIRST, G. **Natural language processing for social media**. [S.l.]: Springer, 2015.
- FÁVERO, E. M. D. B.; CASANOVA, D.; PIMENTEL, A. R. SE³M: A model for software effort estimation using pre-trained embedding models. **Information and Software Technology**, v. 147, p. 106886, 2022. ISSN 0950-5849. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0950584922000507>.
- FISHER, R. A. **Statistical Methods for Research Workers**. Edinburgh: Oliver and Boyd, 1925.
- FONSECA, C. **Word Embedding: fazendo o computador entender o significado das palavras**. 2021. Disponível em: <https://medium.com/turing-talks/word-embedding-fazendo-o-computador-entender-o-significado-das-palavras-92fe22745057>.
- FREUND, Y.; SCHAPIRE, R. E. A decision-theoretic generalization of on-line learning and an application to boosting. **Journal of Computer and System Sciences**, v. 55, n. 1, p. 119–139, 1997.
- GAMMERMAN, A.; VOVK, V.; VAPNIK, V. Learning by transduction. **arXiv preprint arXiv:1301.7375**, 2013. Originally appeared in UAI 1998.

- GARG, S.; SHARMA, R. K.; LIANG, Y. SimpleTran: Transferring Pre-Trained Sentence Embeddings for Low Resource Text Classification. **CoRR**, arXiv, abs/2004.05119, 2020. Disponível em: <https://arxiv.org/abs/2004.05119>.
- GOLDBERG, Y. **Neural Network Methods in Natural Language Processing**. San Rafael, CA, USA.: Morgan & Claypool, 2017. (Synthesis Lectures on Human Language Technologies). ISBN 978-1-62705-298-6. Disponível em: <https://doi.org/10.1007/978-3-031-02165-7>.
- GOODFELLOW, I. J.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge, MA, USA: MIT Press, 2016. <http://www.deeplearningbook.org>.
- GU, J. *et al.* Recent advances in convolutional neural networks. **Pattern recognition**, Elsevier, v. 77, p. 354–377, 2018.
- HAN, S. C. *et al.* **Understanding Graph Convolutional Networks for Text Classification**. 2022.
- HARARY, F. **Graph Theory**. Addison-Wesley Publishing Company, 1969. (Addison-Wesley series in mathematics). ISBN 9788185015552. Disponível em: <https://books.google.com.br/books?id=QNxgQZQH868C>.
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. **The Elements of Statistical Learning**. 2. ed. New York: Springer, 2009.
- HAWKINS, D. M. **Identification of Outliers**. Dordrecht: Springer, 1980.
- HAYKIN, S. **Redes Neurais: Princípios e Prática**. Bookman Editora, 2001. ISBN 9788577800865. Disponível em: <https://books.google.com.br/books?id=bhMwDwAAQBAJ>.
- HEIMANN, P. *et al.* Graph-based software process management. **International Journal of Software Engineering and Knowledge Engineering**, World Scientific, v. 7, n. 04, p. 431–455, 1997.
- HUBER, P. J.; RONCHETTI, E. M. **Robust Statistics**. 2. ed. Hoboken, NJ: Wiley, 2009.
- IDRI, A.; AMAZAL, F. a.; ABRAN, A. Analogy-based software development effort estimation: A systematic mapping and review. **Information and Software Technology**, v. 58, p. 206–230, fev. 2015. ISSN 0950-5849. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0950584914001815>.
- JAMES, G. *et al.* **An Introduction to Statistical Learning**. New York: Springer, 2013. Disponível em: <https://www.statlearning.com/>.
- JANNACH, D. *et al.* A survey on conversational recommender systems. **ACM Computing Surveys (CSUR)**, ACM New York, NY, USA, v. 54, n. 5, p. 1–36, 2021.
- JOACHIMS, T. Transductive inference for text classification using support vector machines. *In: Proceedings of the 16th International Conference on Machine Learning (ICML)*. Bled, Slovenia: Morgan Kaufmann, 1999. p. 200–209.
- JOANES, D. N.; GILL, C. A. Comparing measures of sample skewness and kurtosis. **Journal of the Royal Statistical Society: Series D (The Statistician)**, v. 47, n. 1, p. 183–189, 1998.
- JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics and Speech Recognition: United State**. Upper Saddle River, N.J: Pearson, 2000. ISBN 978-0-13-095069-7.

JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing (3rd ed. (draft))**. 2019. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>.

KIPF, T. N.; WELLING, M. **Semi-Supervised Classification with Graph Convolutional Networks**. 2017.

KOCAGUNELI, E. *et al.* Exploiting the essential assumptions of analogy-based effort estimation. **IEEE Transactions on Software Engineering**, IEEE, v. 38, n. 2, p. 425–438, 2011.

KOCAGUNELI, E. *et al.* Exploiting the essential assumptions of analogy-based effort estimation. **IEEE Transactions on Software Engineering**, v. 38, p. 425–438, 2012. ISSN 00985589.

KONDOR, R. I.; LAFFERTY, J. Diffusion kernels on graphs and other discrete input spaces. *In: Proceedings of the 19th International Conference on Machine Learning (ICML)*. [s.n.], 2002. p. 315–322. Disponível em: <https://people.cs.uchicago.edu/~risi/papers/diffusion-kernels.pdf>.

KUHN, M.; JOHNSON, K. **Applied Predictive Modeling**. New York: Springer, 2013.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **nature**, Nature Publishing Group, v. 521, n. 7553, p. 436, 2015.

LIN, Y. *et al.* **BertGCN: Transductive Text Classification by Combining GCN and BERT**. 2022.

LIU, X. *et al.* Tensor graph convolutional networks for text classification. **Proceedings of the AAAI Conference on Artificial Intelligence**, v. 34, n. 05, p. 8409–8416, Apr. 2020. Disponível em: <https://ojs.aaai.org/index.php/AAAI/article/view/6359>.

LIU, Z.; LIN, Y.; SUN, M. Word Representation. *In: LIU, Z.; LIN, Y.; SUN, M. (Ed.). Representation Learning for Natural Language Processing*. Singapore: Springer, 2020. p. 13–41. ISBN 9789811555732. Disponível em: https://doi.org/10.1007/978-981-15-5573-2_2.

MALEKZADEH, M. *et al.* Review of graph neural network in text classification. *In: IEEE. 2021 IEEE 12th annual ubiquitous computing, electronics & mobile communication conference (UEMCON)*. [S.l.], 2021. p. 0084–0091.

MANIKAVELAN, D.; PONNUSAMY, R. Software cost estimation by analogy using feed forward neural network. *In: Information Communication and Embedded Systems (ICICES), 2014 International Conference on*. Chennai, India: IEEE, 2014. p. 1–5. Disponível em: <https://doi.org/10.1109/ICICES.2014.7033820>.

MANNING, C. D.; SCHÜTZE, H. **Foundations of Statistical Natural Language Processing**. Cambridge, MA: MIT Press, 1999. ISBN 9780262133609.

MENZIES, T. *et al.* Selecting best practices for effort estimation. **IEEE Transactions on Software Engineering**, IEEE, v. 32, n. 11, p. 883–895, 2006.

MIKOLOV, T. *et al.* Efficient Estimation of Word Representations in Vector Space. **arXiv:1301.3781 [cs]**, set. 2013. Disponível em: <http://arxiv.org/abs/1301.3781>.

MONTGOMERY, D. C.; RUNGER, G. C. **Applied Statistics and Probability for Engineers**. 7. ed. Hoboken, NJ: Wiley, 2018. ISBN 9781119400363.

MUKAKA, M. M. A guide to appropriate use of correlation coefficient in medical research. **Malawi Medical Journal**, v. 24, n. 3, p. 69–71, 2012. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC3576830/>.

NADKARNI, P. M.; OHNO-MACHADO, L.; CHAPMAN, W. W. Natural language processing: an introduction. **Journal of the American Medical Informatics Association**, BMJ Group BMA House, Tavistock Square, London, WC1H 9JR, v. 18, n. 5, p. 544–551, 2011.

NASEEM, U. *et al.* A Comprehensive Survey on Word Representation Models: From Classical to State-of-the-Art Word Representation Language Models. **ACM Transactions on Asian and Low-Resource Language Information Processing**, v. 20, n. 5, p. 74:1–74:35, jun. 2021. ISSN 2375-4699. Disponível em: <https://doi.org/10.1145/3434237>.

OLIVEIRA, B. S. N. *et al.* Processamento de linguagem natural via aprendizagem profunda. **Sociedade Brasileira de Computação**, 2022.

OPENAI. **GPT-5 mini: modelo de linguagem de grande porte da OpenAI API**. 2025. <https://platform.openai.com/docs/models/gpt-5-mini>. Acessado em: 18 nov. 2025.

OSBORNE, J. W. Notes on the use of data transformations. **Practical Assessment, Research & Evaluation**, v. 8, n. 6, 2002. Disponível em: <https://openpublishing.library.umass.edu/pare/article/id/1455/>.

O'SHEA, K.; NASH, R. An introduction to convolutional neural networks. **arXiv preprint arXiv:1511.08458**, 2015.

POPESCU, M.-C. *et al.* Multilayer perceptron and neural networks. **WSEAS Transactions on Circuits and Systems**, World Scientific and Engineering Academy and Society (WSEAS) Stevens Point . . . , v. 8, n. 7, p. 579–588, 2009.

RADFORD, A. *et al.* Improving language understanding by generative pre-training. 2018.

RADFORD, A. *et al.* Language models are unsupervised multitask learners. **OpenAI blog**, v. 1, n. 8, p. 9, 2019.

RASCHKA, S. Model evaluation, model selection, and algorithm selection in machine learning. **arXiv preprint arXiv:1811.12808**, 2018. Disponível em: <https://arxiv.org/abs/1811.12808>.

RAZALI, N. M.; WAH, Y. B. Power comparisons of shapiro–wilk, kolmogorov–smirnov, lilliefors and anderson–darling tests. **Journal of Statistical Modeling and Analytics**, v. 2, n. 1, p. 21–33, 2011.

RODRIGUES, P. G. **Extração de Características em Interfaces Cérebro-Máquina Utilizando Métricas de Redes Complexas**. 02 2018. Tese (Doutorado), 02 2018.

ROUSSEEUW, P. J.; LEROY, A. M. **Robust Regression and Outlier Detection**. New York: John Wiley & Sons, 1987. ISBN 9780471852338.

RUBIN, K. S. **Essential Scrum: A Practical Guide to the Most Popular Agile Process**. Boston: Addison-Wesley, 2012. ISBN 978-0137043293.

SALEHINEJAD, H. *et al.* Recent advances in recurrent neural networks. **arXiv preprint arXiv:1801.01078**, 2017.

SHEPPERD, M. Software project economics: A roadmap. *In: 2007 Future of Software Engineering*. IEEE Computer Society, 2007. p. 304–315. ISBN 0769528295. Disponível em: <https://doi.org/10.1109/FOSE.2007.23>.

SHIVHARE, J.; RATH, S. K. Software effort estimation using machine learning techniques. *In: Proceedings of the 7th India Software Engineering Conference*. New York, NY, USA: Association for Computing Machinery, 2014. (ISEC '14). ISBN 9781450327763. Disponível em: <https://doi.org/10.1145/2590748.2590767>.

- STERNBERG, R. J. Component processes in analogical reasoning. **Psychological review**, American Psychological Association, v. 84, n. 4, p. 353, 1977.
- SUN, S. *et al.* Patient Knowledge Distillation for BERT Model Compression. **CoRR**, abs/1908.09355, 2019. Disponível em: <http://arxiv.org/abs/1908.09355>.
- TAWOSI, V.; MOUSSA, R.; SARRO, F. Agile effort estimation: Have we solved the problem yet? insights from a replication study. **IEEE Transactions on Software Engineering**, v. 49, p. 2677–2697, 2022. Disponível em: <https://api.semanticscholar.org/CorpusID:254717340>.
- TRIOLA, M. F. **Elementary Statistics**. 13. ed. Boston, MA: Pearson, 2018. ISBN 9780134462455. Disponível em: <https://www.pearson.com/us/higher-education/product/Triola-Elementary-Statistics-13th-Edition/9780134462455.html>.
- TUKEY, J. W. **Exploratory Data Analysis**. Reading, MA: Addison-Wesley, 1977. ISBN 978-0201076165.
- VAPNIK, V. **Statistical Learning Theory**. Wiley, 1998. (A Wiley-Interscience publication). ISBN 9788126528929. Disponível em: <https://books.google.com.br/books?id=RWrIkQEACAAJ>.
- VASWANI, A. *et al.* Attention Is All You Need. **Computing Research Repository**, abs/1706.03762, 2017. Disponível em: <http://arxiv.org/abs/1706.03762>.
- WESTFALL, P. H. Kurtosis as peakedness, 1905–2014. r.i.p. **The American Statistician**, v. 68, n. 3, p. 191–195, 2014.
- WILCOX, R. **Introduction to Robust Estimation and Hypothesis Testing**. 3. ed. Amsterdam: Academic Press, 2012. ISBN 9780123869838.
- WILLMOTT, C. J.; MATSUURA, K. Advantages of the mean absolute error (mae) over the root mean square error (rmse) in assessing average model performance. **Climate Research**, v. 30, p. 79–82, 2005.
- WILSON, C. *et al.* User interactions in social networks and their implications. In: **Proceedings of the 4th ACM European conference on Computer systems**. [S.l.: s.n.], 2009. p. 205–218.
- WOLPERT, D. H. Stacked generalization. **Neural Networks**, v. 5, n. 2, p. 241–259, 1992.
- WU, Z. *et al.* A comprehensive survey on graph neural networks. **IEEE transactions on neural networks and learning systems**, IEEE, v. 32, n. 1, p. 4–24, 2020.
- XIAO, T.; ZHU, J. **Introduction to Transformers: an NLP Perspective**. 2023.
- YAO, L.; MAO, C.; LUO, Y. **Graph Convolutional Networks for Text Classification**. 2018.
- ZHANG, D. *et al.* Chinese comments sentiment classification based on word2vec and SVMperf. **Expert Systems with Applications**, Pergamon, v. 42, n. 4, p. 1857–1863, 3 2015. ISSN 0957-4174. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0957417414005508>.
- ZHANG, S. *et al.* Graph convolutional networks: a comprehensive review. **Computational Social Networks**, Springer, v. 6, n. 1, p. 1–23, 2019.
- ZHONG, H. *et al.* How does nlp benefit legal system: A summary of legal artificial intelligence. **arXiv preprint arXiv:2004.12158**, 2020.
- ZHOU, D. *et al.* Learning with local and global consistency. In: **Advances in Neural Information Processing Systems (NeurIPS 16)**. [s.n.], 2003. p. 321–328. Disponível em: <https://proceedings.neurips.cc/paper/2506-learning-with-local-and-global-consistency.pdf>.

ZHU, X. **Semi-Supervised Learning Literature Survey**. [S.l.], 2005. Disponível em: https://pages.cs.wisc.edu/~jerryzhu/pub/ssl_survey.pdf.

ZHU, X.; GHAMRANI, Z. **Learning from Labeled and Unlabeled Data with Label Propagation**. [S.l.], 2002. Disponível em: <https://mlg.eng.cam.ac.uk/zoubin/papers/CMU-CALD-02-107.pdf>.

ZHU, X.; GHAMRANI, Z.; LAFFERTY, J. D. Semi-supervised learning using gaussian fields and harmonic functions. *In: Proceedings of the 20th International Conference on Machine Learning (ICML)*. Washington, DC: [s.n.], 2003. p. 912–919. Disponível em: <https://dl.acm.org/doi/10.5555/3041838.3041953>.

GLOSSÁRIO

Transformer modelo de deep learning autosuficiente, que avalia suas entradas gerando uma representação de dados como saída. 28, 29

embedding abreviação de word embeddings. 26–28, 30, 34, 39, 84, veja *word embedding*.

overfitting quando um modelo de aprendizado de máquina se ajusta muito bem aos dados de treinamento, mas tem desempenho ruim em dados novos devido à falta de generalização (Cawley; Talbot, 2010). 54, 57

token instância ou sequência de caracteres de um documento, agrupados como uma unidade semântica a ser processada. 29

word embedding termo usado para descrever a representação de palavras para análise de texto. 26

BertGCN modelo que combina *embedding Bidirectional Encoder Representations from Transformers* (BERT) com redes convolucionais em grafos (GCNs) para aproveitar a rica representação de texto do BERT e a estrutura dos grafos. 3, 4, 14, 15, 34, 35, 55, 56, 59, 60, 66–70, 72–76